

AutoSaddler: 基于智能体执行迹的持久更新自动框架优化

Sungho Park^{1*} Wonjoong Kim^{2*} Rongyuan Tan^{3*} Jue Zhang^{4†} Wook-Shin Han^{1†}
Pengfei Gao⁴ Chanyoung Park² Yongqiang Yao⁴ Rao Fu⁴ Elsie Nallipogu⁴
Qingwei Lin⁴ Saravan Rajmohan⁴ Dongmei Zhang⁴

¹POSTECH ²KAIST ³Southern University of Science and Technology ⁴Microsoft
juezhang@microsoft.com wshan@dblab.postech.ac.kr

摘要

大语言模型智能体在长时程任务上仍不可靠，局部的小失败会在长时间交互中不断累积，最终导致整体任务失败。尽管外部框架能显著提升鲁棒性，但框架设计仍是一个手动且昂贵的过程，需要在提示、工具配置和控制逻辑的庞大空间中搜索。本文提出 **AutoSaddler**，一个自动框架优化框架，将框架改进形式化为离线学习问题，并利用小批量中的失败信号迭代更新框架。**AutoSaddler** 结合了失败迹诊断、将框架视为代码的结构化补丁生成，以及基于验证的更新选择。在 GAIA2、SWE-Bench Pro 和 Terminal-Bench 2.0 上的实验表明，**AutoSaddler** 相较于对应的基础框架显著提升了智能体性能，分别取得了 9.0、9.6 和 10.0 个百分点的提升。消融研究进一步表明，有效的框架优化得益于三个要素：深度调试而非浅层反思、有针对性的修改而非无约束编辑、以及泛化感知的选择而非轨迹特定的修复。这些结果共同表明，自动框架优化是通往更高性能和更可靠智能体系统的一条有前景的路径。项目网站和代码将在 <https://aka.ms/AutoSaddler-website> 提供。

1 引言

大语言模型（Large Language Models）在过去几年中能力迅速提升。尽管如此，它们仍表现出“锯齿状智能”：在某些任务上表现强劲，而在其他看似密切相关甚至更简单的任务上表现不佳。这种不均衡性为智能体应用带来了根本性的可靠性挑战，尤其是在自主、多步骤和长时程场景中，成功依赖于在众多连续决策中保持持续的能力。为此，一条新兴的研究路线致力于在大语言模型周围构建外部框架层，以使智能体行为在长时程任务上更加鲁棒 [61, 3, 5, 31, 21]。经验证据表明，精心设计的框架能显著提升智能体性能 [21]。

然而，手动调整工具框架耗时且难以扩展。它需要探索庞大的设计空间，包括提示词规范、工具配置以及其他系统级选择。此外，评估每个候选工具框架的成本高昂，因为智能体可能需要执行许多步骤才能明确任务成功或失败。分析由此产生的长程轨迹本身并非易事，通常需要大量人工努力。这些挑战催生了对自动化工具框架优化方法的强烈需求。* 本工作于微软实习期间完成。† 通讯作者。

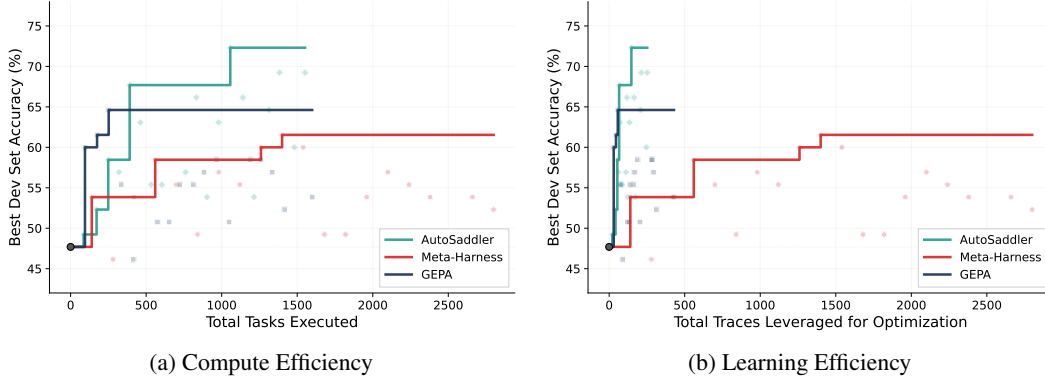


图 1：GAIA2 上优化性能与效率的比较。(a) AutoSaddler 以 $\sim 1,000$ 次总任务执行达到 **72.3%** 的开发集准确率，而 GEPA 和 Meta-Harness 尽管消耗了 $\sim 2,800$ 次任务执行，却分别止步于 **64.6%** 和 **61.5%**。(b) 以用于优化的执行回放次数衡量，AutoSaddler 仅消耗 147 条轨迹即达到最佳性能，比 Meta-Harness (1,400 条轨迹) 少 $\sim 10\times$ 条。

当切换到新的底层大语言模型或将智能体部署到新领域时，能够高效调整工具框架的自动化优化方法。

本文将自动工具框架优化形式化为离线学习问题，并引入 **AutoSaddler** 框架，该框架利用训练任务批次中的失败信号迭代优化工具框架。在每次迭代中，首先从累积的探索历史中构建候选工具框架，并在当前小批量训练样本上对其进行评估。随后诊断失败的智能体轨迹，并利用所得洞见指导补丁生成，将工具框架本身视为代码。对于每次被接受的工具框架更新，进一步在验证集上评估其泛化性。这些组件共同使 **AutoSaddler** 能够以有原则且可扩展的方式优化工具框架，同时促进超越已观测训练样本的泛化更新。

我们在三个智能体基准上评估了 AutoSaddler：GAIA2 [10]、SWE-Bench Pro [9] 和 Terminal-Bench 2.0 [28]。在这三个基准上，AutoSaddler 均显著提升了长程智能体性能。它在 GAIA2 上比对应的基础框架高出 **9.0** 个百分点，在 SWE-Bench Pro 上高出 **9.6** 个百分点，在 Terminal-Bench 2.0 上高出 **10.0** 个百分点，并分别比最强的自动化基线高出 **7.4**、**4.4** 和 **6.7** 个百分点。消融实验和定性分析进一步支持了 AutoSaddler 的设计理念，表明有效的框架优化依赖于三个关键要素。第一，深入诊断是必要的，因为长程失败需要深度调试，而不是对智能体执行失败进行浅层反思。第二，结构化干预很重要，因为庞大且多样的框架空间更倾向于有针对性的修改，而非无约束的编辑。第三，泛化感知的选择至关重要，因为要在任务分布上提升性能，需要保留广泛有用的更新，而不是修复单条轨迹。

我们的主要贡献总结如下：

- 我们将自动框架优化形式化为一个基于提示、工具和中间件的离线学习问题，利用来自智能体执行迹的失败信号。
- 我们提出了 AutoSaddler，它结合了基于证据的诊断、结构化修补和泛化感知的选择，以产生持久的框架更新。
- 我们在三个具有挑战性的智能体基准上进行了大量实验，证明了所提方法在提升长程智能体性能方面的有效性。

2 相关工作

自动提示优化。基于大语言模型的系统的自动优化在提示优化领域已被广泛研究，包括无梯度搜索 [62]、文本梯度和小批量方法 [36]、历史感知和进化优化 [47, 14, 1]、规划-

基于搜索 [43]、编程抽象 [18]、自动微分式优化 [50] 以及贝叶斯多提示微调 [32]。尽管 AutoSaddler 借鉴了这一系列工作，但框架优化比提示优化更为广泛：长时程轨迹通常需要比简单反思更深入的诊断，且搜索空间涵盖提示、工具和运行时控制逻辑。

自进化智能体与基于经验的改进。从失败轨迹中学习与基于经验的智能体改进 [60] 和自进化智能体 [2] 相关，其中经验被用于构建库 [6]、创建工具 [37, 45] 以及构建记忆、技能或知识库 [33, 19, 29]。与这些通常在线持续学习的方法不同，我们专注于离线框架优化，以实现跨环境的泛化。

一些先前的工作进一步强调自指性，即元智能体是否与任务智能体一致，如达尔文-哥德尔机 [54, 55]、哥德尔智能体 [49] 和赫胥黎-哥德尔机 [42]。相比之下，我们不施加此类约束，因为我们的目标是在有限的展开预算下改进任务智能体的外部框架，而非建模自指动态。

智能体系统与框架优化。基于大语言模型系统的外部层自动优化建立在一系列先前工作的基础上，包括大语言模型推理超参数调优 [41, 11]、工作流优化 [56, 64, 44] 以及通过将系统本身视为代码进行系统优化 [59, 16]。最近，这一系列工作已扩展到优化完整的智能体框架 [22, 25, 34, 40]。一项突出的工作是元框架 [22]，它同样采用编码智能体来构建端到端的优化流程。相比之下，我们更强调结构化修补和系统性的训练流程，该流程明确考虑了泛化性。近期同时期的研究³进一步通过可观测性驱动的演化 [23]、诊断与弱点引导的修复 [7, 52]、贝叶斯配置搜索 [38]、历史经验重校准 [13]、自监督回顾性优化 [35]、面向泛化的约束演化 [57]、经验驱动的测试时自适应 [17] 以及框架与模型权重的联合优化 [15, 8] 扩展了自动框架优化。总体而言，这些工作探索了框架优化的互补方面，而 AutoSaddler 强调深入诊断、结构化干预和泛化感知选择的联合作用。

智能体迹失败诊断与修复。具有复杂工具交互的长时程智能体迹推动了失败诊断研究 [58, 51, 26, 12, 53]。这些方法主要针对单迹诊断，并通常假设迹能容纳于大语言模型上下文窗口内，这限制了它们在超长迹和小批量诊断中的应用。本文转而基于克劳德智能体软件开发工具包（Claude Agent SDK, CA-SDK）[4]，利用其文件访问和上下文管理能力。近期工作还研究了用于修复失败运行的迹级干预 [27, 63]。与这类任务特定的“热修复”不同，本文目标是产生持久的测试框架修复，以改善未来行为。

大语言模型驱动的进化搜索。本文的候选选择方法利用大语言模型基于探索历史从先前候选中选择和重组补丁，与近期大语言模型增强的进化算法 [1, 30, 39, 46, 24] 一致。本文以基于 CA-SDK 的候选选择实例化这一思想，将专门的大语言模型驱动进化算法留作未来工作。

3 预备知识

智能体任务与执行迹。本文考虑需要多步推理、重复工具使用和环境交互的智能体任务。设 $x \in \mathcal{X}$ 为从任务分布 $\mathcal{T}$ 中采样的任务描述，并假设 $\mathcal{X}$ 中的任务实例是无状态且独立的。

以测试框架参数 θ 为条件，在任务 x 上执行任务智能体诱导出一个随机执行过程。本文记作 $(\tau, \hat{y}) \sim P_\theta(\cdot \mid x)$ ，其中 τ 为执行迹， $\hat{y}$ 为最终输出。等价地，测试框架 H_θ 定义了条件执行分布 $P_\theta(\tau, \hat{y} \mid x)$ 。这种随机性涵盖了大语言模型采样、工具使用决策和环境交互中的随机性。

³ 这些工作出现在本工作准备的最后阶段或随后的同行评审期间。

优化空间：智能体框架。尽管框架一词常被通俗地用来指代大语言模型之外所有有助于有效智能体行为的组成部分，但其精确范围在社区中仍在演变 [61]。在本工作中，遵循 LangChain[21] 中的框架工程实践，我们关注三类框架参数，不考虑其他组件，如记忆或技能管理，因为我们的设定假设任务在很大程度上是无状态且相互独立的。形式上，我们将优化空间定义为：

$$\theta = (\theta_{\text{prompt}}, \theta_{\text{tool}}, \theta_{\text{middleware}}) \in \Theta, \quad (1)$$

其中 θ_{prompt} 指定指令和系统提示， θ_{tool} 定义可用工具及其接口， $\theta_{\text{middleware}}$ 捕获运行时控制逻辑，包括钩子和智能体循环行为。

目标：预算约束优化。我们的种群目标是在目标任务分布上最大化期望任务性能。设 $\mu(\cdot, y^*)$ 为任务级度量，如准确率或通过/失败成功率，用于评估最终输出 $\hat{y}$ 与黄金答案 y^* 的对比。由于执行框架化的大语言模型智能体可能是随机的，我们定义

$$J(\theta) = \mathbb{E}_{(x, y^*) \sim \mathcal{T}} \mathbb{E}_{(\tau, \hat{y}) \sim P_{\theta}(\cdot | x)} [\mu(\hat{y}, y^*)], \quad \theta^* = \arg \max_{\theta \in \Theta} J(\theta),$$

其中 θ^* 表示优化后的框架。

在实践中， $J(\theta)$ 不可直接观测，每次执行消耗一次采样。给定采样预算 K ，AutoSaddler 在该预算内探索的候选框架集合 $\mathcal{V}_K \subset \Theta$ 上进行搜索。对于有限数据集 D ，我们使用经验估计

$$\hat{J}_D(\theta) = \frac{1}{|D|} \sum_{(x_i, y_i^*) \in D} \frac{1}{R_i} \sum_{r=1}^{R_i} \mu(\hat{y}_{i,r}, y_i^*), \quad (\tau_{i,r}, \hat{y}_{i,r}) \sim P_{\theta}(\cdot | x_i),$$

其中 R_i 是任务 i 的重复执行次数。AutoSaddler 在 rollout 预算内评估的候选项中，返回开发集经验得分最高的候选项：

$$\hat{\theta}_{\text{AS}} = \arg \max_{\theta \in \mathcal{V}_{K, \text{dev}}} \hat{J}_{D_{\text{dev}}}(\theta).$$

最终报告的测试性能为 $J_{D_{\text{test}}}(\hat{\theta}_{\text{AS}})$ 。

4 提出的方法：AutoSaddler

本文提出 AutoSaddler，一个用于大语言模型智能体自动优化执行框架的迭代式框架。首先概述工作流程，然后详细描述每个关键组件。

概述。本文将执行框架优化形式化为离线学习问题。该形式化反映了实际场景：智能体执行框架通常在开发阶段调优，然后部署到生产环境。由于训练可能涉及大量任务，且基于 rollout 的评估成本高昂，本文采用小批量训练范式，遵循标准机器学习实践和先前关于自动提示优化的工作 [36, 1]。

具体而言，将任务集 $\mathcal{X}$ 划分为训练集、开发集和测试集，分别记为 D_{train} 、 D_{dev} 和 D_{test} 。如图 2 所示，每次迭代 n 首先在小批量 $B_n \subset D_{\text{train}}$ 上评估当前执行框架 H_n （参数为 θ_n ）。随后进入诊断-修补会话，分析小批量中失败的执行迹，并推导结构化补丁 $\Delta \theta_n$ ，生成更新后的执行框架 H' （ $\theta_n = H_n + \Delta \theta_n$ ）。修补后的执行框架在同一小批量上进行验证（步骤 3）；若 $J_{B_n}(H'_n) > J_{B_n}(H_n)$ ，则认为该补丁为小批量改进。满足该条件时，进一步在 D_{dev} 上评估 H'_n ，以估计更新是否泛化到观察小批量之外（步骤 4）。

无论补丁是否被接受，反思会话都会比较补丁前后的执行迹；记录已修复、退化、仍失败和仍通过的案例（步骤 5）；并将得到的经验教训和分数存储在 **EvoDAG** 中，这是一个有向无环图，用于跟踪执行框架更新的演化历史（步骤 6）。随后，演化会话利用 **EvoDAG** 提出下一个候选执行框架 H_{n+1} （步骤 7）。当 rollout 预算 K 耗尽后，AutoSaddler 返回得分最高的候选项

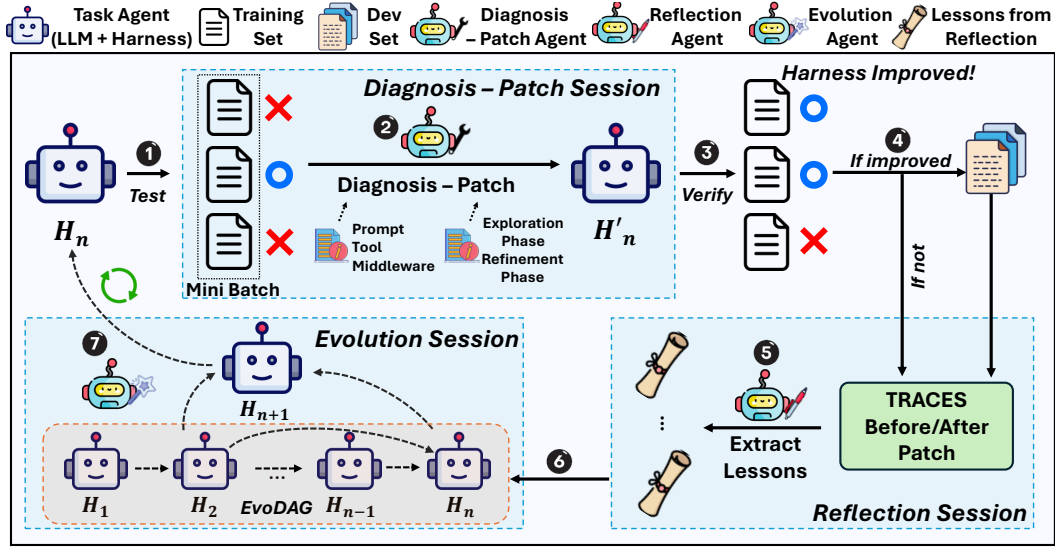


图 2: AutoSaddler 概览。迭代优化循环：当前测试框架在迷你批次上进行测试，跨测试框架组件进行诊断和修补，验证改进效果，然后进行反思以将经验提取到 EvoDAG 中，从而指导下一个测试框架的演化。

在所有候选对象上评估 D_{dev} 得到的经验开发集分数。选定的测试框架随后在留出测试集上评估一次，并且不再使用测试反馈进行修改。

该工作流程模拟了标准的迷你批次学习，但将其适配于文本测试框架优化。诊断-修补-验证步骤（2 和 3）在文本梯度下扮演反向传播的角色：由于文本错误信号不像数值梯度那样自动检查，每次更新都需要显式的假设生成、测试框架干预和经验验证。EvoDAG 和演化会话通过累积反思信号来执行历史感知的测试框架演化，扮演类似优化器的角色。我们在附录 A 中详细阐述了这种类比及其差异。

诊断-修补会话。在此会话期间，来自迷你批次的执行轨迹（包括成功和失败的运行）被传递给诊断-修补智能体，用于故障诊断和补丁生成。我们不将诊断与补丁生成分离，允许智能体在生成补丁时充分利用诊断过程中收集的上下文信息。我们的诊断程序设计为有原则且基于证据的。除了每个故障轨迹外，我们还向智能体提供测试框架代码库 θ_n 和结构化指导，使其能够逐步检索相关轨迹细节，从而缓解长智能体轨迹带来的长上下文挑战。基于这些证据，智能体识别可疑的根本原因，考虑替代假设，并提出补丁生成建议。

补丁生成同样具有结构化特征。具体而言，智能体生成一个结构化补丁 $\Delta \theta_n$ ，该补丁针对一个或多个测试框架组件。我们不允许对完整测试框架代码库进行无约束的编辑，而是仅暴露实现测试框架功能逻辑的源文件，同时禁止访问与评估或基准数据相关的代码。我们进一步将补丁空间划分为三个类别：提示、工具和中间件，每个类别对应智能体测试框架的不同层次。表 1 列举了每个类别的子类型及其描述。

我们进一步将这些补丁类型划分为两个更高层次的组：能力补丁和引导补丁。能力补丁修改可执行代码或编排逻辑，而引导补丁由文本编辑组成，不改变底层代码，包括对提示、工具描述和钩子提醒文本的修改。由于这两组补丁可能以不同方式影响测试框架，我们引入了分阶段补丁调度，类似于标准模型训练中的学习率调度。在该调度中，优化从能力补丁阶段开始，然后过渡到引导补丁阶段。关于分阶段补丁调度的更多细节见附录 A，诊断和补丁生成指令见附录 P。

表 1: 采用的补丁类别, 子类型标记为能力补丁 (C) 或引导补丁 (S)。

Category	Subtype	Description
Prompt Patch	Prompt Rule Addition (S)	Add or modify behavioral rules in the system prompt.
	Prompt Rule Modification (S)	Revise existing prompt rules to resolve conflicts or sharpen guidance.
Tool Patch	New Tool Addition (C)	Add a new tool when no existing tool supports the required action.
	Argument Modification (C)	Add or fix tool parameters to enhance filtering or selection capabilities.
	Implementation Fix (C)	Fix bugs or extend internal tool functionality to produce correct results.
	Tool Description Fix (S)	Modify tool docstrings to avoid tool misuse.
Middleware Patch	PreToolUse Hook (S)	Inject a just-in-time reminder before a specific tool call.
	Infrastructure Change (C)	Modify agent configuration, iteration budget, or environment settings.
	Agent Loop Logic Change (C)	Add preprocessing steps or budget reminders to the agent loop.

反思会话。在评估修补后的测试框架 H'_n 于同一小批量数据, 并可选地在开发集上评估后, 我们进入反思会话, 从当前测试框架更新中提取经验教训。具体而言, 反思智能体被指示比较应用补丁前后智能体在小批量数据上的性能。所得结果分为四类: “已修复”、“性能退化”、“仍然失败”和“仍然通过”。对于每一类, 我们提供针对性的自我反思问题, 以引出关于补丁为何有效、它解决了哪些失败模式、为何发生性能退化、为何补丁不足或补丁是否产生任何效果的洞见。智能体在反思期间也可以检查迹细节。此外, 如果补丁触发了在开发集上的扩展评估, 我们额外提示对开发集性能进行反思, 特别强调补丁是否以及如何在小批量数据之外泛化。这些经验教训连同补丁描述、小批量结果和评估度量结果 $\mu(H'_n)$, 作为节点级属性存储在 EvoDAG 中, 以促进测试框架演化。关于反思指令的更多细节见附录 P。

演化会话。EvoDAG 是一个有向无环图 $\mathcal{G} = (V, E)$, 作为优化过程的累积记忆。每个节点 $v_n \in V$ 对应一个先前探索过的测试框架, 并标注了其相关的经验教训和性能信号。每条有向边 $e \in E$ 表示父测试框架与其后代之间的差异 $\Delta \theta$ 。

在演化会话期间, 演化智能体查阅完整的 EvoDAG 以合成新的测试框架 H_{n+1} 。智能体不是仅从 H'_n 继续, 而是可以组合来自任何先前探索过的测试框架子集的元素, 以图中存储的累积经验教训为指导。这种合并操作类似于演化搜索, 使框架能够通过重组优化历史中不同谱系的成功组件来逃离局部最优。所得测试框架 H_{n+1} 随后在下一代中的新小批量数据上进行评估。演化指令的更多细节见附录 P。

设计理由。总体而言, AutoSaddler 围绕有效自动优化框架的三个需求进行设计: 深入诊断、结构化干预和泛化感知选择。诊断-补丁会话通过将更新建立在长时程执行迹和框架代码库的显式分析之上, 支持深入诊断, 从而实现深度调试而非浅层反思。补丁分类法和分阶段调度通过将优化限制为对提示、工具和中间件的针对性更改, 而非无约束编辑, 支持结构化干预。反思与进化会话通过结合小批量验证、可选的开发集评估和基于 EvoDAG 的记忆, 支持泛化感知选择, 使得保留的更新更有可能泛化到单条轨迹之外。这些组件共同将框架优化转变为一个迭代过程: 诊断失败、应用针对性更新, 并保留更有可能泛化的更改。

5 实验

5.1 实验设置

基准与基础框架系统。我们在三个不同的基准上评估 AutoSaddler: GAIA2 [10]、SWE-Bench Pro (SBP) [9] 和 Terminal-Bench 2.0 (TB2) [28]。GAIA2 评估

表 2: GAIA2 上的测试集 Pass@1 结果, 报告为三次运行的平均值 $\pm$ 标准差。括号中的数值表示任务数量; 粗体表示最佳结果。

Harness (Type)	GAIA2 Universe (Pass@1)			Avg.
	21 (107)	22 (112)	27 (81)	
Default Agent (Manual)	54.8 $\pm$ 4.8	51.5 $\pm$ 4.9	52.7 $\pm$ 4.3	53.0 $\pm$ 1.5
GEPA (Auto)	60.1 $\pm$ 3.9	47.9 $\pm$ 3.4	56.4 $\pm$ 0.7	54.6 $\pm$ 2.5
Meta-Harness (Auto)	53.0 $\pm$ 1.1	51.5 $\pm$ 5.2	56.0 $\pm$ 0.7	53.2 $\pm$ 2.2
AutoSaddler (Auto)	61.4 $\pm$ 2.4	60.7 $\pm$ 2.4	64.6 $\pm$ 3.1	62.0 $\pm$ 1.2
w/o In-depth Diagnosis (Auto)	56.7 $\pm$ 4.2	57.1 $\pm$ 3.9	60.1 $\pm$ 5.0	57.8 $\pm$ 3.8
w/o Structured Intervention (Auto)	58.9 $\pm$ 2.5	53.3 $\pm$ 7.6	59.3 $\pm$ 3.3	56.9 $\pm$ 3.8
w/o Generalization-Aware Selection (Auto)	53.3 $\pm$ 1.9	44.9 $\pm$ 6.7	54.7 $\pm$ 4.0	50.6 $\pm$ 4.0

表 3: SWE-Bench Pro 和 Terminal-Bench 2.0 上的测试集 Pass@1 结果, 报告为三次运行的平均值 $\pm$ 标准差。括号中表示任务数量; 粗体表示每列最佳结果。

SWE-Bench Pro (Pass@1)					Terminal-Bench 2.0 (Pass@1)	
Harness (Type)	Ansible (96)	Flipy (85)	Element-web (56)	Avg.	Harness (Type)	Test-Split (40)
SWE-agent (Manual)	40.6 $\pm$ 1.9	31.0 $\pm$ 3.5	41.1 $\pm$ 1.7	37.3 $\pm$ 4.8	Terminus 2 (Manual)	40.0 $\pm$ 0.0
GEPA (Auto)	50.0 $\pm$ 1.3	32.2 $\pm$ 1.4	45.2 $\pm$ 0.6	42.5 $\pm$ 1.2	Terminus KIRA (Manual)	47.5 $\pm$ 2.5
Meta-Harness (Auto)	36.9 $\pm$ 2.9	31.3 $\pm$ 1.7	38.7 $\pm$ 0.8	35.3 $\pm$ 2.0	GEPA (Auto)	42.5 $\pm$ 2.5
AutoSaddler (Auto)	58.0 $\pm$ 1.8	36.5 $\pm$ 1.8	43.5 $\pm$ 1.6	46.9 $\pm$ 1.8	Meta-Harness (Auto)	43.3 $\pm$ 5.8
					AutoSaddler (Iter2) (Auto)	45.0 $\pm$ 0.0
					AutoSaddler (Iter34) (Auto)	50.0 $\pm$ 0.0

在模拟智能手机环境中评估通用助手能力, 该环境涵盖 10 个不同的宇宙, 每个宇宙模拟一个人物的日常数字生活。我们使用 GAIA2 提供的默认基于 ReAct 的智能体作为基础框架。SBP 针对来自热门仓库的企业级软件工程任务, 我们使用 SWE-agent [48] 作为基础框架。最后, TB2 包含 89 个跨系统管理、机器学习和网络安全等领域的真实任务; 对于 TB2, 我们采用 Terminus 2 作为基础框架。

实现与基线方法。我们使用克劳德代理软件开发工具包 (Claude Agent SDK, CA-) 在自动鞍座 (AutoSaddler) 中实现三个代理 (即诊断补丁代理、反思代理和进化代理)

软件开发工具包 (SDK) [4]。详细的智能体指令见附录 P, EvoDAG 的实现细节见附录 B。本文将 AutoSaddler 与 GEPA [1] 及 Meta-Harness [22] 进行对比。

GEPA 是一种以提示词为中心的基线方法, 通过迭代优化系统提示词。相比之下, Meta-Harness 将优化目标从提示词转向智能体框架, 采用极简系统设计, 并采用 CA-SDK 进行端到端系统优化。由于 Meta-Harness 仅评估了 TB2, 我们将 GEPA 适配到 GAIA2 和 TB2, 并将 Meta-Harness 适配到 GAIA2; 适配细节见附录 B。

实验方案与数据划分。除非另有说明, 所有优化方法均使用克劳德·欧普斯 4.6 (Claude Opus 4.6) 作为底层大语言模型 (LLM), 并采用默认端点设置。为评估泛化能力, 我们确保训练集、开发集和测试集包含来自不同任务组的任务。例如, 在 SBP 中, 训练集由来自 qutebrowser 的任务组成; 开发集包含来自 Vuls 和 NodeBB 的任务; 测试集则包含来自 Ansible、Flipy 和 Element-web 的任务。这种划分使我们能够跨代码库评估泛化能力。关于所有三个基准的数据划分详情, 请参见附录 B。除非另有说明, 对于每种优化方法, 我们在训练集和开发集上执行一次进化运行, 并在测试集上评估时进行三次重复运行。⁴ 我们报告 Pass@1 成功率作为评估度量。

5.2 主要结果

我们在表 2 中报告了 GAIA2 测试集的 Pass@1 成功率，并在 SBP 和 TB2 的测试集中报告了

表 3. 得出两个主要观察结果。首先，AutoSaddler 发现了更有效的约束装置
优于所有三个基准中对应的基础框架。Specifically, AutoSaddler improves over the default agent on GAIA2 by **+9.0 pp** (53.0% $\rightarrow$ 62.0%), over SWE-agent on SBP by **+8.4 pp** (37.3% $\rightarrow$ 46.9%), and over Terminus 2 on TB2 by **+10.0 pp** (40.0% $\rightarrow$ 50.0%). Second, **AutoSaddler 优于自动化基线 GEPA 和 Meta-Harness。**与各基准上最强的自动化基线相比，(54.6% vs. 62.0%), **+6.2 pp** 42.5% vs. 46.9% **+4.4 pp** on TB2 (43.3% vs. 50.0%). On AutoSaddler 在 SBP 的 GAIA2 上取得了 **+7.4 pp** 的提升 ()，以及 TB2，(47.5% vs. 50.0%). AutoSaddler 还超越了手动专家调校的线束，Terminus KIRA [20]，超出 **+2.5 pp** 本文进一步验证了对优化随机性和训练分布偏移的鲁棒性。在 GAIA2 上独立运行的 AutoSaddler 达到 58.6% Pass@1，仍显著高于基础框架和重跑基线（附录 C），而在不同训练集上的优化相比基础框架获得 57.4%，a **+5.9 pp** 的提升（附录 D）。

除跨任务组评估外，本文进一步分析跨模型迁移性。具体而言，本文将 GAIA2 任务代理的底层大语言模型（LLM）从 Opus 4.6 替换为 Haiku 4.5，同时保留使用 Opus 4.6 优化的框架。如附录 E 所示，AutoSaddler 仍比基础框架提升 **+5.6 pp**，表明其具有跨模型迁移的潜力。

为分析优化效率，本文将自动鞍座设计器（AutoSaddler）与自动化基线在 GAIA2 上的优化轨迹进行对比，如图 1 所示；自动鞍座设计器完整搜索轨迹的可视化见附录 J。研究发现，自动鞍座设计器以

大幅减少任务智能体（task-agent）的 rollout 次数。具体而言，AutoSaddler 仅用 72.3% 次 rollout 就达到了 $\sim 1,000$ 的开发准确率，而 GEPA 和 Meta-Harness 尽管消耗了 $\sim 2,800$ 次任务执行，准确率却分别停留在 64.6% 和 61.5%（图 1a）。当以用于学习的 rollout 次数来衡量效率时，对比更加鲜明（图 1b）：AutoSaddler 在消耗 147 次 rollout 后达到最佳开发集分数， $\sim 10\times$ 少于 Meta-Harness 的 (1,400 次 rollout)。在 TB2 上也观察到类似的效率提升，详见附录 H。我们在附录 I 中进一步刻画了端到端优化成本。尽管 AutoSaddler 每个补丁的优化器侧货币成本略高，但其选择性评估策略仅用 391 次任务智能体 rollout 就达到了 67.7% 的开发准确率，已经超过了 Meta-Harness 在 1,400 次 rollout 后达到的 61.5% 峰值。

5.3 消融实验与分析

接下来我们在 GAIA2 上展示消融实验，重点关注 AutoSaddler 背后的三个关键设计原则：深入诊断、结构化干预和泛化感知选择。

研究问题 1：深入诊断能否实现更优的根因识别？我们首先进行消融实验，评估深入诊断是否改善根因识别并最终提升整体性能。具体来说，我们评估一个“无深入诊断”变体，该变体将基于 CA-SDK 的诊断替换为浅层诊断基线：单次大语言模型调用接收执行迹和评估结果，并推断失败原因，这是自动提示优化流程中常用的失败反思策略 [36, 1]。推断出的失败原因随后传回 CA-SDK 用于补丁生成。相比之下，AutoSaddler 的深入诊断主动探索执行迹和源代码以调查失败。

如表 2 所示，移除深度诊断会显著降低 GAIA2 测试集上的性能，将 Pass@1 从 62.0 降至 57.8。深度诊断的益处进一步得到附录 K 中详细的文件访问和工具调用分析的支持：与仅补丁的会话相比，诊断-补丁联合会话在每个优化步骤中平均多调用 6.2 次工具和多访问 5.8 次文件。我们还定量比较了两种设置下训练期间接受的补丁的累积数量，其中接受的补丁定义为在同一小批量上提高性能的补丁；参见附录 K 中的图 10。深度诊断始终产生更多被接受的补丁。到第 1 轮（第 25 次迭代）结束时，差距显著

4 由于 Meta-Harness 本身不支持训练-开发集划分，我们为其提供训练集和开发集的并集，以确保可比的数据暴露。

(13 对 5)，并且这一优势在整个第 2 轮（第 26-50 次迭代）中得以保持。最后，我们在附录 K 中提供了几个定性案例研究，说明浅层诊断与 AutoSaddler 的深度诊断在识别根本原因方面的差异。

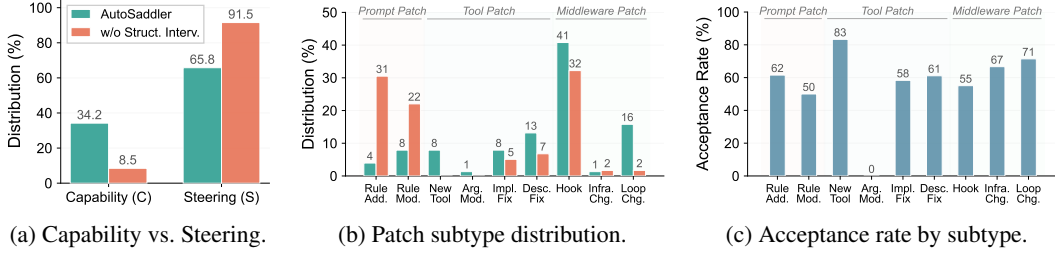


图 3：补丁类型分布与接受情况：AutoSaddler 与无结构化干预的对比。在没有结构化干预的情况下，补丁集中在转向（91.5%）上，而 AutoSaddler 产生了涵盖提示、工具和中间件编辑的均衡组合。

RQ2：结构化干预如何影响补丁多样性和有效性？ 为了评估结构化干预的有效性，我们考虑了一个消融设置，即无结构化干预，在该设置中我们移除了所提出的补丁分类法和分阶段补丁调度。该消融使智能体在没有明确搜索空间边界的情况下进行无约束编辑（如 Meta-Harness 中那样）。如表 2 所示，移除结构化补丁会显著降低 GAIA2 测试集上的 Pass@1，从 62.0% 降至 56.9%。这一结果表明，将框架优化构建为有针对性的结构搜索对于实现有意义的性能提升至关重要。附录 F 中的细粒度消融进一步表明，仅移除分阶段补丁调度会将 Pass@1 从 60.7% 降至 54.8%，而移除完整的结构化干预会进一步将其降至 53.3%。

进一步分析表明，在无结构化干预（w/o Structured Intervention）设置下，性能下降源于有偏的补丁探索和补丁多样性的降低，尽管允许无约束的补丁编辑。如图 3a 和图 3b 所示，在没有结构化干预的情况下，系统高度集中于转向补丁（Steering patches）（91.5%），这些补丁主要对应简单的文本编辑，而很少探索更高价值的干预措施，如基础设施或工具改进。不同补丁类型的效果差异进一步由图 3c 中补丁子类型的接受率分解所支持。以能力为中心、非提示层的补丁，即新工具（83%）、循环变更（71%）和基础设施变更（67%），获得了最高的接受率。然而，在无结构化干预设置下，这些关键补丁类型仅占生成补丁的 4%；相比之下，AutoSaddler 通过结构化补丁将其占比提高到 25% 以上。AutoSaddler 在进化过程中发现的这些类别的代表性补丁见附录 L。综合这些结果表明，结构化干预鼓励智能体超越提示层编辑，探索能力层面的改进。此外，附录 G 显示，能力补丁的修复率与转向补丁相当（55% 对 58%），但引起的回归显著更少（8% 对 17%），表明更新更持久。

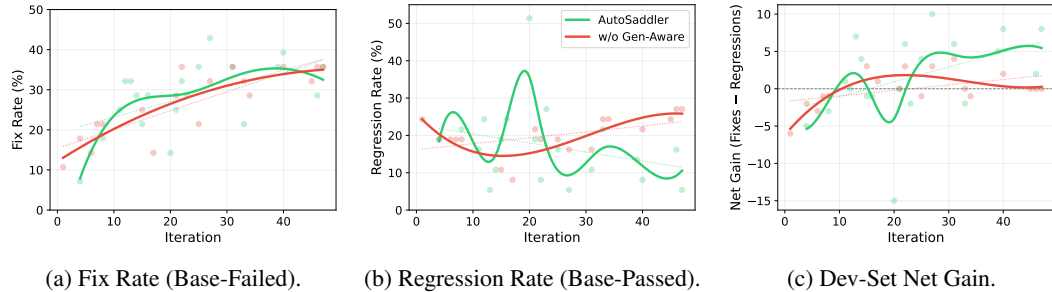


图 4：开发集上各迭代的性能比较。AutoSaddler 在保持与无泛化感知选择（w/o Generalization-Aware Selection）消融相当的修复率的同时，维持了更低的回归率，从而产生持续的正净增益，而消融未能保持这一增益。

研究问题 3：泛化感知选择如何防止过拟合？最后，我们进行消融实验，以考察泛化感知优化如何缓解对训练集的过拟合，并减少在未见场景上的性能退化。具体而言，“无泛化感知选择”消融实验移除了反思会话和开发集评估，迫使优化仅依赖训练集执行结果。如表 2 所示，移除该组件显著降低了 GAIA2 测试集性能，将 Pass@1 从 62.0% 降至 50.6%。这是所有组件消融中性能下降幅度最大的。附录 F 中的细粒度消融进一步表明，移除开发集过滤使 Pass@1 从 60.7% 降至 50.0%，而额外移除带有 EvoDAG 的反思则进一步降至 44.9%。

为了量化该机制对开发集行为的影响，图 4 在开发集上追踪了两种设置，并将净性能分解为修复率和退化率。修复率定义为初始基础框架失败场景上的成功率，退化率定义为基础框架通过场景上的失败率。值得注意的是，两种设置实现了相似的修复率（图 a），表明性能差距并非主要由问题解决能力的差异驱动。相反，关键差异在于退化率（图 b）。尽管两条曲线在优化过程中均有波动，AutoSaddler 整体呈现下降的退化趋势（-0.24 个百分点/迭代），而消融设置则呈现上升趋势（+0.16 个百分点/迭代）。

我们进一步考察了消融设置在第 21 次迭代时出现的退化尖峰（图 b）（8% → 22%）。在第 20 次迭代时，消融设置引入了一个新工具 `send_progress_message_to_user`，并修改了广泛使用的 `send_message_to_user` 工具的钩子，以强制将智能体重定向到该新工具。由于缺乏反思来评估附带损害，这一过于宽泛的补丁被保留下来，干扰了智能体在许多无关开发场景中的行为。有趣的是，同样的失败模式——在高频工具上设置超出其预期范围的过于宽泛的钩子——也出现在 AutoSaddler 的第 4 次迭代中（见图 12b 及附录 M 中的扩展讨论）。然而，由于 AutoSaddler 中的反思机制，此类导致退化的补丁被阻止。这一直接对比表明，泛化感知选择通过识别和过滤范围过大的补丁，有助于维持在未见场景上的性能提升。

最后，图 4c 展示了由此产生的净增益，表明 AutoSaddler 在第 20 次迭代后持续超越消融版本。这表明 AutoSaddler 能够从早期迭代的回归挑战中恢复，并通过从具体优化经验中抽象出可泛化的原则，在后续迭代中实现更稳定的改进。关于原则泛化和定性案例研究的更多讨论见附录 M。

6 结论

本文提出了 AutoSaddler，一个自动化的框架优化框架，将面向大语言模型智能体的框架改进形式化为基于执行迹的离线学习问题。AutoSaddler 结合了深度故障诊断、跨提示、工具和中间件的结构化补丁生成，以及通过验证和基于 EvoDAG 的进化实现的泛化感知更新选择。因此，它产生持久的框架更新，而非针对特定轨迹的修复。在 GAIA2、SWE-Bench Pro 和 Terminal-Bench 2.0 上，AutoSaddler 分别将对应基础框架的性能持续提升 9.0、9.6 和 10.0 个百分点，并在每个基准上分别以 7.4、4.4 和 6.7 个百分点的优势超越了最强的自动化基线。这些结果表明，有效的框架优化需要深度调试、有针对性的干预以及显式的泛化选择，将自动框架优化定位为构建更高效、更可靠的长时程智能体系统的一个有前景的方向。

7 致谢

我们感谢匿名审稿人提出的建设性意见与建议，这些意见帮助提升了本文的清晰度和质量。我们感谢 Bo Qiao 在搭建实验所用计算服务器方面给予的慷慨帮助。我们进一步感谢 GAIA2、SWE-Bench Pro、Terminal-Bench 2.0、SWE-agent、Terminus 2、GEPA 和 Meta-Harness 的创建者与维护者，他们公开了各自的基准、系统和工具，使本文的实验得以开展。

参考文献

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alex Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [2] Huan ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru WANG, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. Survey Certification.
- [3] Anthropic. Effective harnesses for long-running agents. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>, 2025. Accessed 2026-04-21.
- [4] Anthropic. Agent sdk overview - claude code docs. <https://code.claude.com/docs/en/agent-sdk/overview>, 2026. Accessed 2026-04-21.
- [5] Anthropic. Harness design for long-running application development. <https://www.anthropic.com/engineering/harness-design-long-running-apps>, 2026. Accessed 2026-04-21.
- [6] Zhicheng Cai, Xinyuan Guo, Yu Pei, Jiangtao Feng, Jiangjie Chen, Ya-Qin Zhang, Wei-Ying Ma, Mingxuan Wang, and Hao Zhou. FLEX: continuous agent evolution via forward learning from experience. *CoRR*, abs/2511.06449, 2025.
- [7] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable llm agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026.
- [8] Zhengyu Chen, Teng Xiao, Huaisheng Zhu, Yige Yuan, Luan Zhang, and Jingang Wang. Co-harness: Co-evolving harnesses and model weights for llm agents. *arXiv preprint arXiv:2607.22688*, 2026.
- [9] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can AI agents solve long-horizon software engineering tasks? *CoRR*, abs/2509.16941, 2025.
- [10] Romain Froger, Pierre Andrews, Matteo Bettini, Amar Budhiraja, Ricardo Silveira Cabral, Virginie Do, Emilien Garreau, Jean-Baptiste Gaya, Hugo Laurençon, Maxime Lecanu, Kunal Malkan, Dheeraj Mekala, Pierre Ménard, Gerard Moreno-Torres Bertran, Ulyana Piterbarg, Mikhail Plekhanov, Mathieu Rita, Andrey Rusakov, Vladislav Vorotilov, Mengjue Wang, Ian Yu, Amine Benhalloum, Grégoire Mialon, and Thomas Scialom. Gaia2: Benchmarking LLM agents on dynamic and asynchronous environments. *CoRR*, abs/2602.11964, 2026.
- [11] Jia Fu, Xiaoting Qin, Fangkai Yang, Lu Wang, Jue Zhang, Qingwei Lin, Yubo Chen, Dongmei Zhang, Saravan Rajmohan, and Qi Zhang. Autorag-hp: Automatic online hyper-parameter tuning for retrieval-augmented generation. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, Findings of ACL, pages 3875–3891. Association for Computational Linguistics, 2024.
- [12] Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. Who is introducing the failure? automatically attributing failures of multi-agent systems via spectrum analysis. *arXiv preprint arXiv:2509.13782*, 2025.

- [13] Hanghui Guo, Weijie Shi, Zhangze Chen, Shengxiang Xu, Yishu Wang, Yimei Zhang, Wangze Ni, Jia Zhu, and Shimin Di. Drevo: Distilling recalibrated historical experience for harness self-evolution. *arXiv preprint arXiv:2607.26722*, 2026.
- [14] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [15] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. Sia: Self improving ai with harness & weight updates. *arXiv preprint arXiv:2605.27276*, 2026.
- [16] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.
- [17] Yue Huang, Wenjie Wang, Han Bao, Yuchen Ma, Xiaonan Luo, Yi Nian, Haomin Zhuang, Zheyuan Liu, Yue Zhao, and Xiangliang Zhang. Memoharness: Agent harnesses that learn from experience. *arXiv preprint arXiv:2607.14159*, 2026.
- [18] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.
- [19] Shashank Kirtania, Param Biyani, Priyanshu Gupta, Yasharth Bajpai, Roshni Iyer, Sumit Gulwani, and Gustavo Soares. Improving language agents through BREW. *CoRR*, abs/2511.20297, 2025.
- [20] KRAFTON AI and Ludo Robotics. Terminus-kira: Boosting frontier model performance on terminal-bench with minimal harness. <https://github.com/krafton-ai/kira>, 2026.
- [21] LangChain. Improving deep agents with harness engineering. <https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering>, 2026. Accessed 2026-04-21.
- [22] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.
- [23] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.
- [24] Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z Pan, et al. Evox: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026.
- [25] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness. *arXiv preprint arXiv:2603.03329*, 2026.
- [26] Guoqing Ma, Jia Zhu, Hanghui Guo, Weijie Shi, Jiawei Shen, Jingjiang Liu, and Yidan Liang. Automatic failure attribution and critical step prediction method for multi-agent systems based on causal inference. *arXiv preprint arXiv:2509.08682*, 2025.
- [27] Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Dover: Intervention-driven auto debugging for LLM multi-agent systems. In *The Fourteenth International Conference on Learning Representations*, 2026.

- [28] Mike A. Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, Estefany Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis-Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *CoRR*, abs/2601.11868, 2026.
- [29] Nous Research. Hermes agent — the agent that grows with you. <https://hermes-agent.nousresearch.com/>, 2026. Accessed 2026-04-21.
- [30] Alexander Novikov, Ngân Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. *CoRR*, abs/2506.13131, 2025.
- [31] OpenAI. Harness engineering: leveraging codex in an agent-first world. <https://openai.com/index/harness-engineering/>, 2026. Accessed 2026-04-21.
- [32] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 9340–9366. Association for Computational Linguistics, 2024.
- [33] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [34] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*, 2026.
- [35] Wenbo Pan, Shujie Liu, Chin-Yew Lin, Jingying Zeng, Xianfeng Tang, Xiangyang Zhou, Yan Lu, and Xiaohua Jia. Evolving agents in the dark: Retrospective harness optimization via self-preference. *arXiv preprint arXiv:2606.05922*, 2026.
- [36] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 7957–7968. Association for Computational Linguistics, 2023.
- [37] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution. *CoRR*, abs/2505.20286, 2025.
- [38] Biswa Sengupta and Jinhua Wang. Harbor: Automated harness optimization. *arXiv preprint arXiv:2604.20938*, 2026.

- [39] Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025. Accessed 2026-04-21.
- [40] Varun Ursekar, Apaar Shanker, Veronica Chatrath, Sam Denton, et al. Vero: An evaluation harness for agents to optimize agents. *arXiv preprint arXiv:2602.22480*, 2026.
- [41] Chi Wang, Xueqing Liu, and Ahmed Hassan Awadallah. Cost-effective hyperparameter optimization for large language model generation inference. In Aleksandra Faust, Roman Garnett, Colin White, Frank Hutter, and Jacob R. Gardner, editors, *International Conference on Automated Machine Learning, 12-15 November 2023, Hasso Plattner Institute, Potsdam, Germany*, Proceedings of Machine Learning Research, pages 21/1–17. PMLR, 2023.
- [42] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [43] Xinyuan Wang, Chenxi Li, Zhen Wang, Fan Bai, Haotian Luo, Jiayou Zhang, Nebojsa Jojic, Eric P. Xing, and Zhiting Hu. Promptagent: Strategic planning with language models enables expert-level prompt optimization. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [44] Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. Evoagentx: An automated framework for evolving agentic workflows. In Ivan Habernal, Peter Schulam, and Jörg Tiedemann, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025 - System Demonstrations, Suzhou, China, November 4-9, 2025*, pages 643–655. Association for Computational Linguistics, 2025.
- [45] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-swe-agent: Can software engineering agents self-evolve on the fly? *CoRR*, abs/2511.13646, 2025.
- [46] Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Shuo Chen, Zhankui He, Naveen Sachdeva, Isabella Ye, Weili Wang, et al. Pacevolve: Enabling long-horizon progress-aware consistent evolution. *arXiv preprint arXiv:2601.10657*, 2026.
- [47] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [48] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
- [49] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 27890–27913. Association for Computational Linguistics, 2025.
- [50] Mert Yüsekşen, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text. *Nature*, 639, 2025.
- [51] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. Agentracer: Who is inducing failure in the LLM agentic systems? *CoRR*, abs/2509.03312, 2025.
- [52] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026.

- [53] Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search. *arXiv preprint arXiv:2510.10581*, 2025.
- [54] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [55] Jenny Zhang, Bingchen Zhao, Wannan Yang, Jakob N. Foerster, Jeff Clune, Minqi Jiang, Sam Devlin, and Tatiana Shavrina. Hyperagents. *CoRR*, abs/2603.19461, 2026.
- [56] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.
- [57] Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, et al. Harnesscompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses. *arXiv preprint arXiv:2608.01918*, 2026.
- [58] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.
- [59] Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 60315–60335. PMLR / OpenReview.net, 2024.
- [60] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: LLM agents are experiential learners. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, pages 19632–19642. AAAI Press, 2024.
- [61] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, et al. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*, 2026.
- [62] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- [63] Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, et al. Where llm agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*, 2025.
- [64] Runchuan Zhu, Bowen Jiang, Lingrui Mei, Fangkai Yang, Lu Wang, Haoxiang Gao, Fengshuo Bai, Pu Zhao, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. Adaptflow: Adaptive workflow optimization via meta-learning. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 3287–3302. Association for Computational Linguistics, 2025.

Traditional Mini-batch Training	AutoSaddler Stage	Interpretation
Step 1: Sample mini-batch	❶ Sample mini-batch for evaluating current harness	Same role
Step 2: Run forward pass	❶ Execute the current harness	The agent rollout is the forward computation. It produces both a final answer and a trajectory.
Step 3: Compute loss	❶ Record outcomes and traces	The task outcome provides the sparse metric signal; the trace provides evidence needed to explain the outcome.
Step 4: Backpropagate gradients	❷ Diagnose and patch + ❸ verify on the same mini-batch	AutoSaddler has no numerical gradient; it constructs a textual update through hypothesis generation on root cause, intervention, and verification.
Step 6: Validate/checkpoint	❹ Evaluate on the dev-set	Dev-set evaluation checks whether the candidate improvement generalizes beyond the mini-batch.
Step 5: Apply optimizer update	❺ Reflect + ❻ store in EvoDAG + ❼ evolve the next harness	The committed update is not merely the raw patch. AutoSaddler updates optimizer state and uses it to synthesize the next candidate harness.
Step 7: Repeat	Next iteration	The evolved harness is evaluated on a fresh mini-batch.

表 4：传统小批量训练与图 2 中 AutoSaddler 循环之间的映射。AutoSaddler 遵循小批量学习的外部结构，但将可微梯度计算替换为基于迹的符号优化。

A AutoSaddler 作为文本框架参数上的小批量学习

本附录阐明图 2 中的 AutoSaddler 循环与标准小批量训练循环之间的关系。这一类比之所以有用，是因为 AutoSaddler 保留了小批量学习的外部结构：它反复在小批量上评估候选系统，提取更新信号，检查更新是否具有泛化性，并利用累积的优化历史来提出下一个候选。然而，这一类比不应被字面理解。与神经参数不同，harness 参数是文本性的、可执行的且不可微分的。因此，AutoSaddler 用基于迹的符号优化过程替代了可微梯度计算。

映射到传统小批量训练。表 4 总结了对应关系。我们将传统小批量训练的阶段记为步骤 1 – 步骤 7，并使用图 2 中的带圈标签来指代 AutoSaddler 的阶段。

为什么反向传播变成了诊断、修补和验证。两种设置之间的核心不匹配在于反向传播的类比物。在神经训练中，一旦计算出损失，反向传播就提供了一个数学上定义的梯度。因此，更新方向相对明确。

在 AutoSaddler 中，一次失败的 rollout 并不能直接指明 harness 应如何改变。根本原因可能在于提示、工具接口、工具实现、中间件钩子或智能体循环。因此，系统必须从迹中推断出根本原因假设，实施有针对性的干预，并经验性地测试该干预是否在同一小批量上改善了行为：诊断 → 修补作为干预 → 同批量验证。

(2)

这一序列扮演了构建可信文本梯度的角色。重要的是，此阶段的修补尚不宜被理解为最终提交的参数更新。相反，它是一种用于测试所推断的根本原因是否合理的干预。这一区别很重要，因为数值梯度源自固定的数学算子，而文本梯度是推断出来的、语义性的且可能出错的。因此，同批量验证是必要的，以避免将未经检验的解释视为可靠的更新方向。

Algorithm 1 AutoSaddler as mini-batch learning over harness parameters

Require: Initial harness H_0 , training set D_{train} , development set D_{dev} , rollout budget K

```
1: Initialize EvoDAG  $G_0$  with  $H_0$ 
2: for iteration  $n = 0, 1, 2, \dots$  until budget  $K$  is exhausted do
3:   Sample mini-batch  $B_n \subset D_{\text{train}}$  ▷ Figure 2: ❶; Step 1
4:   Execute  $H_n$  on  $B_n$ , collecting outcomes and traces  $\tau_n$  ▷ ❶; Steps 2–3
5:   Diagnose failures in  $\tau_n$  and infer root-cause hypotheses ▷ ❷; textual credit assignment
6:   Generate targeted patch  $\Delta\theta_n$ , yielding candidate  $H'_n = H_n + \Delta\theta_n$  ▷ ❷; patch as intervention
7:   Re-evaluate  $H'_n$  on  $B_n$  ▷ ❸; verify textual gradient
8:   if same-batch performance improves then
9:     Evaluate  $H'_n$  on  $D_{\text{dev}}$  ▷ ❹; Step 6 generalization gate
10:  end if
11:  Reflect on before/after traces and evaluation results ▷ ❺; convert evidence into lessons
12:  Update EvoDAG  $G_n$  with patch, metrics, and lessons to obtain  $G_{n+1}$  ▷ ❻; symbolic optimizer state
13:  Evolve next harness  $H_{n+1}$  from  $G_{n+1}$  ▷ ❼; Step 5 committed update
14: end for
15: return best harness selected by development performance
```

为什么优化器更新是反思加上 EvoDAG 演化。第二个不匹配之处涉及优化器更新的概念。在常规训练中，优化器直接修改数值参数。在 AutoSaddler 中，原始补丁只是一个候选变更。在允许此变更影响未来搜索之前，系统必须确定它是否在当前小批量之外仍然有用，以及应保留哪些可复用的经验。

这就是为什么提交的更新更适理解开发集验证 $\rightarrow$ 反思 $\rightarrow$ EvoDAG 更新 $\rightarrow$ 下一个框架的进化。(3)。反思将前后证据提炼为可复用的经验，EvoDAG 将这些经验与补丁描述和性能信号一起存储。进化会话随后利用这种符号优化器状态，在构建下一个候选时选择、修改或重组先前探索过的框架中的组件。从这个意义上说，EvoDAG 充当了一种优化器记忆。类似于动量累积过去的梯度信息以平滑噪声更新并引导未来步骤走向历史上富有成效的方向，EvoDAG 累积过去的补丁、结果和反思，以引导框架进化走向具有泛化效益证据的变更。因此，AutoSaddler 的更新是历史感知和组合式的，而不是单一的局部算术步骤。

关于顺序的说明。上述映射有意交换了步骤 5 和步骤 6 的通常顺序。在标准小批量训练中，通常先应用优化器更新，然后再进行验证或检查点保存。在 AutoSaddler 中，开发集验证先于最终的符号优化器更新，因为 EvoDAG 是优化器状态的一部分。一旦存储了误导性的经验，它就可能影响未来的进化。因此，验证在系统将经验提交给 EvoDAG 并进化 H_{n+1} 之前充当泛化门。

算法视角。算法 1 以算法方式呈现了相同的视角。关键点在于，在 2 – 3 中生成的补丁充当用于测试推断文本梯度的干预。搜索轨迹的最终更新仅在验证、反思和基于 EvoDAG 的进化之后发生。

分阶段补丁调度。我们根据表 1 中补丁对测试框架（harness）所做的更改类型，将其分为两类。能力补丁修改可执行代码或编排逻辑，包括工具实现、工具参数、基础设施设置和智能体循环逻辑。这些补丁可以改变智能体能够执行的动作，或改变测试框架执行这些动作的方式。引导补丁是文本编辑，不改变底层可执行代码，包括对提示、工具描述和钩子提醒文本的修改。这些补丁主要改进智能体如何在现有能力中进行选择，以及如何遵循任务特定的约束。

这种区分类似于但不完全等同于基于梯度的优化中的大学习率步长与小学习率步长。能力补丁通常表现得像较大的步长，因为它们可以引入新功能、改变控制流或改变可用的动作空间。引导补丁通常表现得像较小的步长，因为它们已在建立的能力集内调整智能体的行为。

然而，这种类比只是近似的：提示或钩子编辑有时会产生较大的行为变化，尤其是当它们影响高频决策时。因此，我们将能力-引导区分用作调度启发式，而不是对效果大小的严格保证，并依靠验证和开发集验证来检测过于宽泛或导致回归的更改。

受此类比启发，AutoSaddler 采用两阶段调度，其转换点为 k ：

1. 探索阶段 ($n \leq k$)。搜索优先考虑能力补丁，以解决工具、基础设施和智能体循环行为中的根本性缺口。2. 细化阶段 ($n > k$)。在能力集稳定后，搜索切换到引导补丁以细化行为。

转换点 k 可以直接指定为迭代次数，也可以通过训练轮次数隐式指定：

$$k = E \cdot \left\lceil \frac{|\mathcal{D}_{\text{train}}|}{B} \right\rceil,$$

其中 E 是能力阶段的训练轮次数， B 是小批量大小。与学习率调度一样，将 k 设置得太小可能会对高影响力的能力改进探索不足，而将其设置得太大可能会延迟风险较低的行为细化。在我们的实验中，单个能力阶段训练轮次 ($E = 1$) 效果良好，允许搜索在切换到有针对性的引导调整之前，用高影响力的补丁遍历训练集一次。

要点。 AutoSaddler 可视为对外部化智能体参数进行的小批量离线学习。其设计不同于标准梯度训练，因为框架参数是文本型且不可微的；性能信号稀疏且延迟；长时程迹上的信用分配模糊不清。因此，额外的机制并非偶然。迹收集、诊断、以补丁作为干预、同批次验证、开发集验证、反思、EvoDAG 记忆以及进化，是传统小批量训练中由损失函数、反向传播、优化器状态和检查点紧凑处理的机制的符号化对应物。

B 实验设置的额外细节

EvoDAG 的实现。 EvoDAG 底层的 DAG 可以直接实现，但为智能体提供对 DAG 的有效访问更具挑战性：随着候选者和迹在迭代中不断累积，直接导航 EvoDAG 变得不切实际。我们没有将完整 DAG 序列化到提示中，而是引入了 `evo-dag` 命令行接口 (CLI)，以促进 CA-SDK 与 EvoDAG 之间的交互。该接口提供对 DAG 结构化摘要的按需访问，包括补丁历史、经验教训、场景注册表和框架代码差异。这使得智能体能够在深入特定原始迹或源文件进行详细分析之前，识别相关上下文。完整命令集总结于表 5。

将基线适配到我们的基准。 GEPA 将系统提示视为单一统一字符串。相比之下，GAIA2 中默认的基于 ReAct 的智能体将其系统提示分布在多个部分中，例如核心行为原则、ReAct JSON 工具调用格式以及 Meta 的智能体研究环境 (ARE) 仿真环境指令。这些部分被定义为独立变量，仅在推理时拼接，造成结构不匹配，阻碍了 GEPA 的直接应用。为解决这一不匹配问题，我们将各部分拼接为单一字符串，使用 `===== VAR_NAME =====` 分隔符标记部分边界，并指示 GEPA 的反思大语言模型在提示演化过程中保留这些分隔符。演化后的提示随后沿分隔符拆分，所得各部分传递给 CA-SDK 以更新相应变量。对于 SBP 和 TB2，我们分别采用 SWE-agent 和 Terminus 2 作为基础测试框架；由于这两个智能体均将系统提示定义为单一统一字符串，GEPA 可直接应用而无需进一步修改。

原始 Meta-Harness 假设测试框架包含在单个 Python 脚本中，并要求继承自固定的基础智能体类。然而，在 GAIA2 默认智能体中，测试框架可能分布在整个代码库中。因此，我们扩展了 Meta-Harness 的适配

表 5: evo-dag 命令行界面暴露的命令，分为用于限定范围 EvoDAG 视图的读取操作和用于每个会话节点或场景更新的写入操作。

命令 用途 使用时机		
读取操作		
evo-dag summary	DAG topology, best candidate, all edges	Start of session; quick orientation
evo-dag show history	Full patch history: diffs, reflections, lessons	Deep dive into prior attempts
evo-dag show node <idx>	Node details: scores, intent, verdict, output dirs	Inspecting a specific candidate
evo-dag show edge <parent> <child>	Code diff, per-scenario impacts, files changed	Understanding a specific patch
evo-dag show scenario <id>	Per-scenario history, root causes, attempted fixes	Before diagnosing a failing scenario
evo-dag show current-batch	Current mini-batch scenario IDs and output dirs	Orienting to the current iteration
evo-dag show lineage	DAG lineage with edge types	Understanding branching structure
evo-dag show lessons	Accumulated good/bad patterns	Checking known patterns before patching
写入操作		
evo-dag update-selection	记录选定的父候选者及选择背后的推理 演化会话，父选择之后	
进化有向无环图更新意图	记录目标场景、诊断、方法、修改的文件以及变更摘要	诊断-补丁会话，应用补丁后
进化有向无环图更新反思	记录每个场景的状态（已修复 / 、退化 / 、仍然失败 / still_passing）、根本原因、补丁后解释、预防或下一步以及泛化说明	反思会话，每个场景

通过指示 CA-SDK 在整个仓库中应用原始元测试框架中使用的相同补丁程序，将其提升到仓库级别。这使得整体测试框架优化超越了单文件设置。

基准特定评估协议。对于每个基准，我们遵循其官方评估基础设施，以确保报告的结果与先前的排行榜数据直接可比。

GAIA2。我们使用 are-run 命令通过 ARE 运行默认的基于 ReAct 的智能体，该命令在相应的沙盒化 Universe 中执行每个任务并记录完整轨迹。遵循 GAIA2 协议 [10]，我们使用 Llama-3.3-70B-Instruct 作为评判模型，根据智能体的最终状态判定任务是否成功。

SWE-Bench Pro。遵循 [9] 的官方协议，每个智能体生成的补丁都会在预配置了问题提交版本仓库的逐实例 Docker 镜像中应用。随后使用该实例的失败转通过和通过转通过单元测试对补丁进行验证。仅当所有失败转通过测试均通过且无任何通过转通过测试出现回归时，该任务才被计为已解决。

Terminal-Bench 2.0。遵循 [28]，本文使用 Harbor 在隔离的 Docker 容器中运行每个任务。成功与否由任务特定的测试脚本判定，该脚本检查最终的终端和系统状态。

各基准上的数据划分。为评估优化后的测试框架（harness）能否泛化至优化过程中未见过的任务，本文针对每个基准分别构建训练集、开发集与测试集，如表 6 所示。对于具有自然分组结构的基准，

我们按任务组而非单个任务进行划分。在 GAIA2 中，划分轴是宇宙（Universe），每个宇宙对应一个独特的角色和模拟数字环境。这种设计确保测试集包含优化过程中未见过的任务组，从而比随机任务级划分提供更强分布外泛化测度。在 SWE-Bench Pro 中，划分轴是代码仓库，这也会引起编程语言、代码库结构和问题分布的变化。这种设计确保测试集包含优化过程中未见过的任务组，从而比随机任务级划分提供更强分布外泛化测度。对于 Terminal-Bench 2.0，任务涵盖多个领域，但没有自然的分组轴；因此我们采用均匀随机划分，分为训练集、开发集和测试集。

表 6: 三个基准的数据划分。为评估分布偏移下的泛化能力，我们将每个基准划分为训练集、开发集和测试集，使各集合包含来自不相交任务组的任务——GAIA2 按宇宙（角色）划分，SWE-Bench Pro 按仓库（编程语言）划分——而非随机任务级划分。Terminal-Bench 2.0 仅包含跨多个领域的 89 个任务，且无自然分组轴，因此我们采用均匀随机划分。

Benchmark	Split Axis	Split	Task Group	# Tasks
GAIA2	Universe (persona)	Train	Universe 29	75
		Dev	Universe 30	65
		Test	Universe 21	107
			Universe 22	112
			Universe 27	81
SWE-Bench Pro	Repository (language)	Train	qutebrowser (Python)	79
		Dev	Vuls (Go)	40
			NodeBB (JavaScript)	40
		Test	Ansible (Python)	96
			Flipy (Go)	85
			Element-web (TypeScript)	56
Terminal-Bench 2.0	Random [†]	Train	—	30
		Dev	—	19
		Test	—	40

[†] Only 89 tasks across heterogeneous domains (system administration, machine learning, cybersecurity); no natural axis for distribution-shifted splits exists.

优化预算。对于 AutoSaddler 和 GEPA，我们在 GAIA2 和 SWE-Bench-Pro 上优化 2 个训练轮次，在 Terminal-Bench 2.0 上优化 4 个训练轮次；TB2 上翻倍的预算弥补了其训练集规模约为 GAIA2 一半的不足。对于 Meta-Harness，采用相同的基于训练轮次的预算会产生误导：与执行小批量优化的 AutoSaddler 和 GEPA 不同，Meta-Harness 在全批量机制下运行，因此 2 个训练轮次的预算仅对应两次补丁更新，会不公平地削弱该基线。为确保公平比较，我们改为按训练期间消耗的总任务执行次数匹配优化预算。具体而言，我们在 GAIA2 上训练 Meta-Harness 20 个训练轮次，在 Terminal-Bench 2.0 上训练 15 个训练轮次，在 SWE-Bench-Pro 上训练 8 个训练轮次，确保其在每个基准上的总任务执行次数不少于 AutoSaddler 所使用的次数。

C 对优化随机性的鲁棒性

我们的主要实验对每种方法仅执行一次优化运行，因为端到端框架优化比重复的测试时评估昂贵得多。为评估所观察到的增益是否对随机优化轨迹敏感，我们在 GAIA2 上对 AutoSaddler、GEPA 和 Meta-Harness 进行了额外的独立优化运行。我们使用每个框架三次重复执行，在 Universe 22（最大的留出测试宇宙，包含 112 个场景）上评估每个所得框架。

表 7：对优化随机性的鲁棒性。来自两次独立优化运行的框架在 GAIA2 Universe 22 上的 Pass@1。测试性能报告为三次执行的均值 $\pm$ 标准差。

Method	GAIA2 Universe 22 (112 scenarios; Pass@1)
Default Agent (Manual)	51.5 $\pm$ 4.9
GEPA (Run 1)	47.9 $\pm$ 3.4
GEPA (Run 2)	50.6 $\pm$ 0.5
Meta-Harness (Run 1)	51.5 $\pm$ 5.2
Meta-Harness (Run 2)	51.2 $\pm$ 2.2
AutoSaddler (Run 1)	60.7 $\pm$ 2.4
AutoSaddler (Run 2)	58.6 $\pm$ 0.5

表 7 报告了结果。第二次 AutoSaddler 优化运行达到 58.6% 的 Pass@1，仅比第一次运行低 2.1 个百分点，比默认智能体高 7.1 个百分点。它还分别比独立重新运行的 GEPA 和 Meta-Harness 框架高出 8.0 和 7.4 个百分点。尽管两次优化轨迹不足以对优化器进行全面的统计表征，但独立运行间一致的增益表明 AutoSaddler 的改进并非特定于某条有利的搜索轨迹。

附录 D 对训练分布偏移的鲁棒性

表 8：对训练分布偏移的鲁棒性。当 AutoSaddler 使用不同训练宇宙独立优化时，在 GAIA2 宇宙 22 上的 Pass@1。数值为三次测试执行的均值 $\pm$ 标准差。

Method	GAIA2 Universe 22 (112 scenarios; Pass@1)
Default Agent (Manual)	51.5 $\pm$ 4.9
AutoSaddler (Run 1; Universe 29)	60.7 $\pm$ 2.4
AutoSaddler (Run 2; Universe 29)	58.6 $\pm$ 0.5
AutoSaddler (Different Train Set; Universe 24)	57.4 $\pm$ 2.1

主要的 GAIA2 实验使用宇宙 29 作为训练集来优化框架。为检验所得增益是否关键依赖于这一特定训练分布，我们将宇宙 29 替换为规模相当的宇宙 24，并在保持其余优化和评估协议不变的情况下独立优化 AutoSaddler。所得框架在宇宙 22 上评估三次。

如表 8 所示，在宇宙 24 上优化的框架达到 57.4% 的 Pass@1。尽管使用不同的训练宇宙进行优化，它仍以默认智能体提高 5.9 个百分点，并保持在宇宙 29 上训练的第二次独立 AutoSaddler 运行的 1.2 个百分点以内。这些结果表明，AutoSaddler 能够从不同的人物角色和任务分布中发现有效的框架更新，而非依赖于原始宇宙 29 训练划分的特定属性。

附录 E 对较弱智能体骨干的跨模型可迁移性

虽然表 2 报告了以 Claude Opus 4.6 同时作为优化器和智能体骨干的结果，但一个关键的实用问题是：由强模型优化的框架在部署到较弱任务智能体时是否仍能保持其优势。为验证这一点，我们在 GAIA2 上使用 Claude Haiku 4.5 作为任务智能体骨干，对所有方法进行重新评估，同时保持框架与基于 Opus 优化时产生的框架不变。表 9 显示，AutoSaddler 相较于默认智能体实现了 +5.6 个百分点的整体提升，证明了有效的跨模型迁移能力。值得注意的是，AutoSaddler 在所有评估领域中均一致优于所有基线方法和消融设置。

表 9：跨模型迁移能力。使用 Claude Haiku 4.5 作为智能体骨干，在 GAIA2 测试集上的 Pass@1 成功率，框架由 Claude Opus 4.6 优化。数值为三次运行的平均值 $\pm$ 标准差。

Harness (Type)	GAIA2 Universe (Pass@1)			Avg.
	21 (107)	22 (112)	27 (81)	
Default Agent (Manual)	31.2 $\pm$ 5.9	24.4 $\pm$ 1.4	36.2 $\pm$ 1.9	30.0 $\pm$ 1.8
GEPA (Auto)	33.6 $\pm$ 1.9	24.4 $\pm$ 0.5	38.3 $\pm$ 1.2	31.4 $\pm$ 0.4
Meta-Harness (Auto)	30.8 $\pm$ 0.0	26.8 $\pm$ 4.7	34.2 $\pm$ 4.0	30.2 $\pm$ 2.7
AutoSaddler (Auto)	38.3 $\pm$ 0.0	30.4 $\pm$ 1.8	39.1 $\pm$ 2.6	35.6 $\pm$ 0.8
w/o In-depth Diagnosis (Auto)	36.1 $\pm$ 5.5	19.0 $\pm$ 5.2	34.2 $\pm$ 0.7	29.2 $\pm$ 3.4
w/o Structured Intervention (Auto)	32.1 $\pm$ 3.0	28.0 $\pm$ 4.0	37.4 $\pm$ 5.8	32.0 $\pm$ 3.5
w/o Generalization-aware Selection (Auto)	28.3 $\pm$ 2.9	23.2 $\pm$ 4.1	30.0 $\pm$ 3.8	26.9 $\pm$ 1.3

F 结构化干预与泛化感知选择的细粒度消融

表 10：细粒度消融。GAIA2 领域 22 上的 Pass@1。数值为三次运行的平均值 $\pm$ 标准差。

Method	GAIA2 Universe 22 (112 scenarios; Pass@1)
Default Agent (Manual)	51.5 $\pm$ 4.9
AutoSaddler	60.7 $\pm$ 2.4
w/o Structured Intervention	53.3 $\pm$ 7.6
w/o Phase Scheduling	54.8 $\pm$ 4.4
w/o Generalization-Aware Selection	44.9 $\pm$ 6.7
w/o Dev-Set Filtering	50.0 $\pm$ 5.0

主要消融实验整体移除了 AutoSaddler 的每个设计原则。我们进一步解耦了结构化干预和泛化感知选择背后的各个机制。对于结构化干预，主要的无结构化干预消融同时移除了补丁分类法和分阶段补丁调度。因此，我们引入了一个更细粒度的无分阶段调度变体，该变体保留补丁分类法，仅移除分阶段调度。类似地，主要的无泛化感知选择消融同时移除了开发集过滤和基于 EvoDAG 的反思。我们引入了一个无开发集过滤变体，该变体保留基于 EvoDAG 的反思，但移除开发集过滤。对于每个变体，我们重新运行完整的优化流程，并使用三次重复测试执行在 GAIA2 领域 22 上评估生成的框架。

表 10 展示了结果。仅移除分阶段补丁调度（Phased Patch Scheduling）会使 Pass@1 从 60.7% 降至 54.8%，下降 5.9 个百分点。进一步移除补丁分类法会使性能降至 53.3%，表明补丁空间结构化带来了额外的 1.5 点贡献。对于泛化感知选择，移除开发集过滤会使 Pass@1 从 60.7% 降至 50.0%，而进一步移除带 EvoDAG 的反思（Reflection with EvoDAG）会使性能降至 44.9%。因此，在此比较中开发集过滤贡献了更大的效果，而带 EvoDAG 的反思提供了额外的 5.1 点增益。综合来看，这些更细粒度的消融表明，每个设计原则中的两种机制都对最终测试框架性能有所贡献。

G 补丁持久性分析

理想的测试框架更新不仅应修复促使补丁产生的场景，还应保持原本正确的行为。因此，我们通过比较同一训练小批量上每个生成补丁前后的任务结果来分析补丁持久性。我们将修复率定义为应用补丁后先前失败场景通过的比例，将回归率定义为应用补丁后先前通过场景失败的比例。

我们首先根据补丁分类法使用的三个测试框架组件对补丁进行分组：提示（Prompt）、工具（Tool）和中间件（Middleware）。如表 11 所示，三个类别的修复率相似，为 57 – 59%，而工具补丁的回归率略高，为 19%，相比之下提示和中间件补丁的回归率为 14%。因此，仅凭被编辑的组件并不能揭示清晰的持久性模式。

表 11：按测试框架组件划分的补丁持久性。修复率和回归率通过比较同一训练小批量上补丁前后的结果来衡量。

Patch Category	Fix Rate (%)	Regression Rate (%)
Prompt	59	14
Tool	57	19
Middleware	57	14

这些类别之间区分度较弱的一个原因是它们混合了不同的干预机制。例如，工具补丁既包括可执行变更（如新工具和实现修复），也包括对工具描述的文本修改。类似地，中间件补丁既包括可执行基础设施或智能体循环变更，也包括纯文本的预工具使用钩子。因此，我们使用主论文中更高层的能力-引导分类法对所有生成的补丁进行重新分类。能力补丁修改可执行功能或编排逻辑，而引导补丁在不改变底层可执行能力的情况下修改文本指令。

表 12 揭示了一个更清晰的差异。引导补丁的修复率略高于能力补丁（58% 对 55%），但其回归率是能力补丁的两倍多（17% 对 8%）。因此，能力补丁在实现相当的局部修复率的同时，将回归率降低了 9 个百分点。这一结果表明，能力层面的干预往往产生更持久的更新，而文本引导更容易波及到预期范围之外的场景。这一量化模式也与我们的定性示例一致：范围明确的可执行修复通常解决确定性的能力局限，而过于宽泛的钩子或提示规则可能影响无关场景。

表 12：按干预机制划分的补丁持久性。能力补丁和引导补丁表现出相似的修复率，但引导补丁引发的回归显著更多。

Patch Type	Fix Rate (%)	Regression Rate (%)
Steering Patch	58	17
Capability Patch	55	8

H 在终端基准 2.0 上的优化效率

为了评估自动鞍座优化效率是否超越 GAIA2，我们报告了在终端基准 2.0 上的计算和学习效率，该基准针对一种不同的智能体能力：长时程命令行问题求解。我们遵循与主 GAIA2 实验（第 5 节）相同的协议，将自动鞍座与元框架和 GEPA 进行比较。

图 5 报告了在终端基准 2.0（Terminal-Bench 2.0）上的计算与学习效率。从共同的 52.6% 起点出发，自动鞍座（AutoSaddler）仅经过 31 次任务执行和 12 条利用迹后即达到 **73.7%** 的开发准确率，比元框架（Meta-Harness）的 63.2% 高出 10.5 个百分点，比通用环境感知代理（GEPA）的 57.9% 高出 15.8 个百分点。相对于元框架（98 条迹）的 $\sim 8\times$ 利用迹减少与在通用人工智能助手 2（GAIA2）上观察到的趋势一致，表明自动鞍座的诊断引导修补在以终端为中心的任务上提供了持续的优化效率增益，这些任务的失败主要由工具使用和环境交互错误主导，而非多跳推理。

I 端到端优化成本表征

仅任务代理的展开并不能完全表征自动框架优化的计算成本。除了框架评估之外，每个优化器还可能产生大量的优化器侧

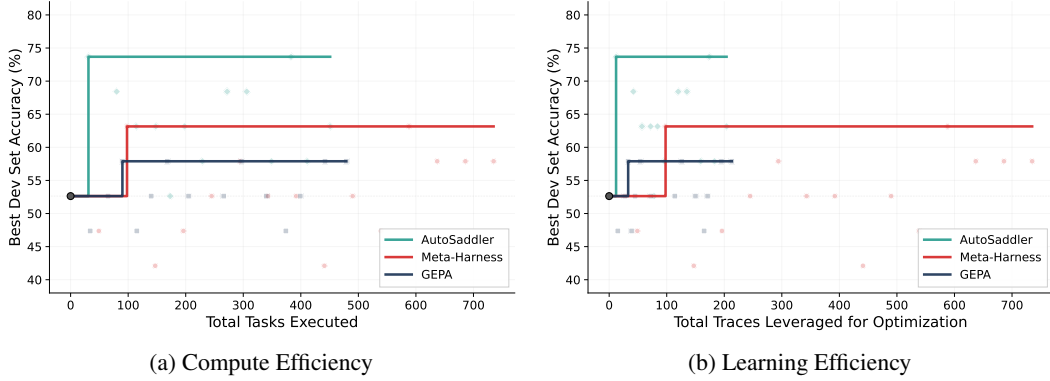


图 5：终端基准 2.0 上优化性能与效率的比较。(a) 自动鞍座仅用 31 次任务执行即达到 73.7% 的开发准确率，而元框架需要 98 次执行才在 63.2% 处趋于平稳，通用环境感知代理在 90 次执行时达到 57.9%。(b) 以用于优化的执行迹数量衡量时，自动鞍座仅利用 12 条迹即达到最佳性能，比元框架（98 条迹）少 $8 \times$ 条以上，同时高出 10.5 个百分点。

用于候选生成、诊断、修补、反思和候选选择的大语言模型（LLM）开销。因此，我们分别剖析了 (i) 优化器侧大语言模型开销和 (ii) 通用人工智能助手 2 上的任务代理评估成本。

优化器侧成本。表 13 报告了生成、拒绝和接受的补丁数量，以及每个生成补丁的平均墙钟时间、货币成本、大语言模型调用次数和令牌使用量。通用环境感知代理的优化器侧成本最低，每个补丁 5.50 美元，但仅搜索系统提示。元框架和自动鞍座则优化更广泛的框架组件，因此提供了更直接的比较。自动鞍座每个生成补丁的成本为 14.56 美元，比元框架多 1.91 美元，而每个补丁需要 533 秒而非 883 秒，对应墙钟时间降低 39.6%。尽管自动鞍座明确执行诊断、结构化补丁生成、反思和进化，这一差异仍然存在。

表 13：通用人工智能助手 2 上的优化器侧成本。运行时间、货币成本、大语言模型调用次数和令牌使用量按每个生成补丁取平均。

Method	Generated Patches	Rejected Patches	Accepted Patches	Wall-Clock Time (s) / Patch	Cost (\$) / Patch	LLM Calls / Patch	Output Tokens / Patch	Cache-Creation Input Tokens / Patch	Cache-Read Input Tokens / Patch
GEPA	76	58	18	386	5.50	21.3	11,416	156,788	280,940
Meta-Harness ⁵	20	—	—	883	12.65	66.9	43,429	250,466	3,528,484
AutoSaddler	39	19	20	533	14.56	76.1	21,464	430,933	3,246,158

任务智能体评估成本。测试框架评估的成本远高于上述优化器侧操作。一次 GAIA2 任务智能体展开平均需要 20.2 次大语言模型调用、550,988 个输入令牌、7,380 个输出令牌以及 203.9 秒的墙钟时间，总结于

表 14。

表 14：每次 GAIA2 展开的平均任务智能体评估成本。

Metric	Average per Rollout
LLM calls	20.2
Input tokens	550,988
Output tokens	7,380
Wall-clock time	203.9 s

⁵ 元测试框架未采用显式的补丁接受门控；因此，拒绝和接受的补丁数量不适用。

各方法的评估协议在调用这一昂贵操作的频率上存在显著差异。元测试框架对每个候选更新评估全部 140 个 GAIA2 训练和开发场景的并集。相比之下，自动鞍座器每次优化迭代评估六个训练场景，并且仅对改善训练小批量的补丁调用 65 个场景的开发集评估。因此，自动鞍座器在 391 次任务智能体展开后达到 67.7% 的开发准确率，已经超过了元测试框架在 1,400 次展开后达到的 61.5% 峰值。因此，虽然自动鞍座器在每个生成补丁上产生略高的优化器侧货币成本，但这一开销被显著更选择性地使用昂贵的任务智能体评估以及在更小的展开预算下更高的性能所抵消。

J 搜索轨迹可视化

为了说明反思和进化会话如何共同塑造搜索轨迹，图 6 可视化了自动鞍座器在 GAIA2 上 50 次迭代（2 个训练轮次）中的开发分数进展。每个节点代表一个在其小批量上被接受并随后在留出开发集上评估的测试框架（51 个总候选中的 21 个）；显示的分值是开发集准确率。实线边表示基础继承（细灰色为顺序继承，粗深色为变基），红色虚线边表示择优合并操作，这些操作导致有向无环图结构而非简单链。

该轨迹呈现出四个阶段。在基础构建阶段（第 0 至第 8 次迭代），演化会话在每一步选择唯一可用的前驱节点，形成一条线性链。在快速改进阶段（第 8 至第 15 次迭代），五次连续接受操作依次线性叠加，在第 13 次迭代达到峰值（67.7%）。有向无环图结构出现在选择性合并与修复阶段（第 16 至第 27 次迭代）：第 20 次迭代因高频工具上的钩子导致性能灾难性回退至 33.8% 后，演化会话重新基于第 13 次迭代，并将第 13 次和第 14 次迭代中经过验证的修复补丁择优合并到第 21 次和第 22 次迭代，同时回退通过反思识别出的有害变更。第 27 次迭代通过线性链继承了这些精心挑选的补丁，并达到全局峰值 72.3%。在巩固与变基阶段（第 31 至第 47 次迭代），所有分支均基于第 27 次迭代进行变基。当第 46 次迭代过度累积了来自第 40/44/45 次迭代的八个择优钩子和规则，导致下降 12.3 个百分点时，第 47 次迭代的演化会话将其诊断为集体引导开销，并裁剪为仅保留四个保守补丁，恢复到 69.2%。总体而言，该轨迹凸显了 AutoSaddler 如何利用反思引导的变基和选择性合并来保留有益变更、丢弃有害变更，并在测试框架搜索过程中从性能回退中恢复。

K 研究问题 1 的补充分析

深度诊断的工具调用与文件访问开销 AutoSaddler 的一个核心设计选择是深度诊断阶段，其目的是对失败轨迹进行比“无深度诊断”消融设置中的浅层反思更彻底的调查。为了验证该阶段确实进行了实质性更深入的分析，而非仅仅增加表面开销，我们实证测量了 AutoSaddler 相比消融设置所额外消耗的工具调用和文件访问次数。在 AutoSaddler 中，Claude 智能体软件开发工具包同时执行诊断和补丁修复，而在消融变体中，该工具包仅负责补丁修复。因此，两者之间的差距直接反映了可归因于深度诊断的调查工作量。

统计工具调用次数较为直接，因为每次调用克劳德代理软件开发工具包（Claude Agent SDK）工具都会被记录。然而，准确统计文件访问次数并非易事，因为克劳德代理软件开发工具包偶尔会将工作委派给子代理，而子代理内部发出的工具调用不会记录在主跟踪中。为确保两种变体测量的一致性，我们按以下标准统计主代理的文件访问次数：

- **通过专用文件工具进行文件访问。** 克劳德代理软件开发工具包专用文件处理工具（即读取、写入、编辑、多重编辑和搜索）的调用次数。
- **通过命令行命令进行文件访问。** 读取或操作文件内容的 shell 命令执行次数，包括 cat、wc、grep 以及打开文件的 Python 调用（例如包含 open() 调用的 python3 或 python 脚本）。

相同的统计规则同时应用于自动鞍座（AutoSaddler）和消融变体，从而能够直接比较调查工作量。

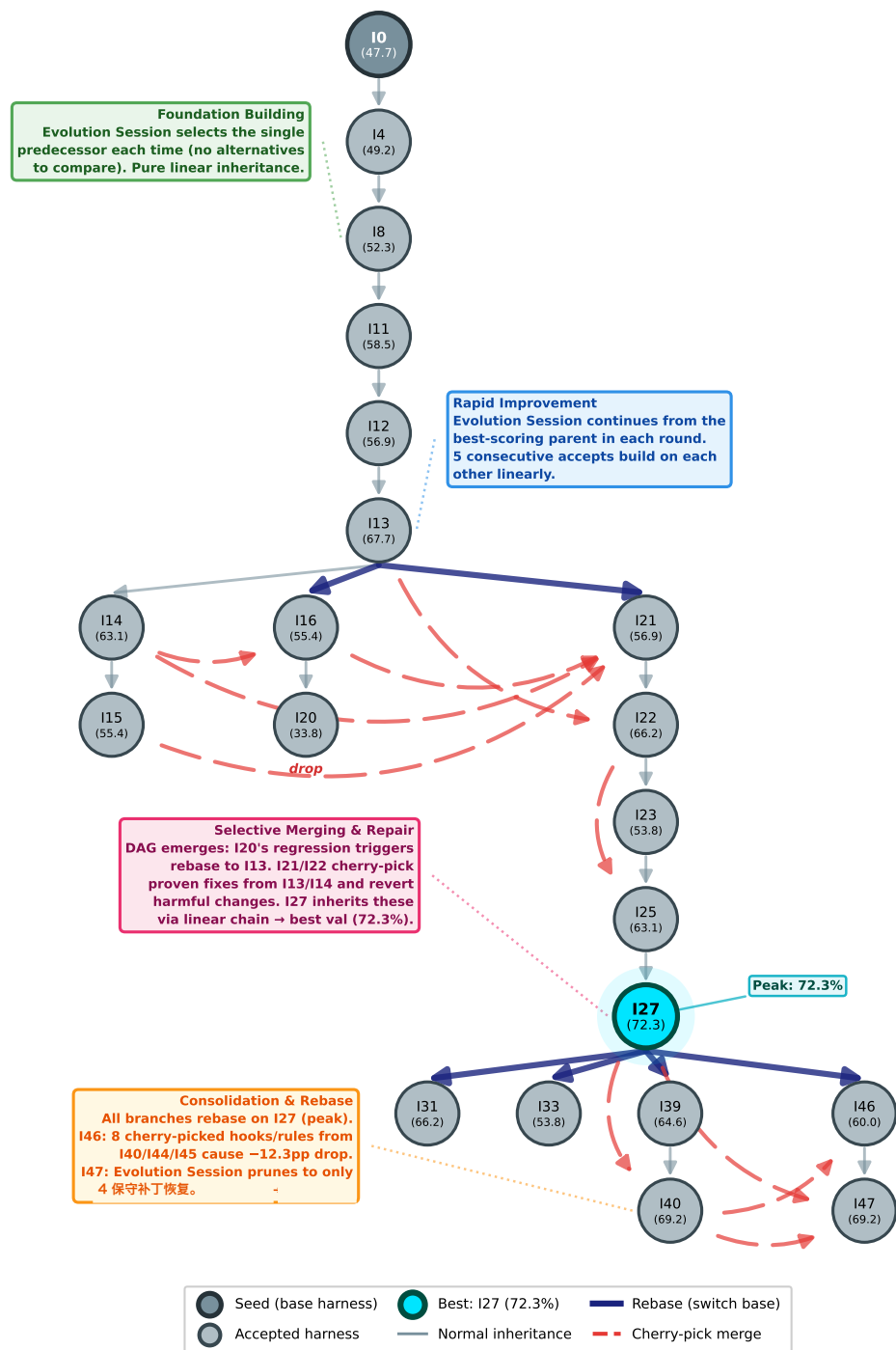


图 6：自动鞍座作为进化有向无环图（EvoDAG）的进化搜索轨迹。四个阶段（基础、快速改进、选择性合并和巩固）在迭代 27 达到全局开发集峰值 72.3%，摆脱了迭代 20 的回归和局部最优。

表 15 报告了 AutoSaddler 与“无深入诊断”消融设置在每次优化步骤中的平均工具调用次数和文件访问次数。AutoSaddler 平均执行 69.7 次工具调用和 45.5 次文件访问，而消融变体分别为 63.5 次和 39.7 次，对应每次步骤多出 6.2 次工具调用和 5.8 次文件访问。

表 15: AutoSaddler 及其无深入诊断消融变体在每次优化步骤中的平均工具调用次数和文件访问次数。

Method	Avg. Tool Calls	Avg. File Accesses
w/o In-Depth Diagnosis	63.5	39.7
AutoSaddler	69.7	45.5
Δ (Attributable to In-Depth Diagnosis)	+6.2	+5.8

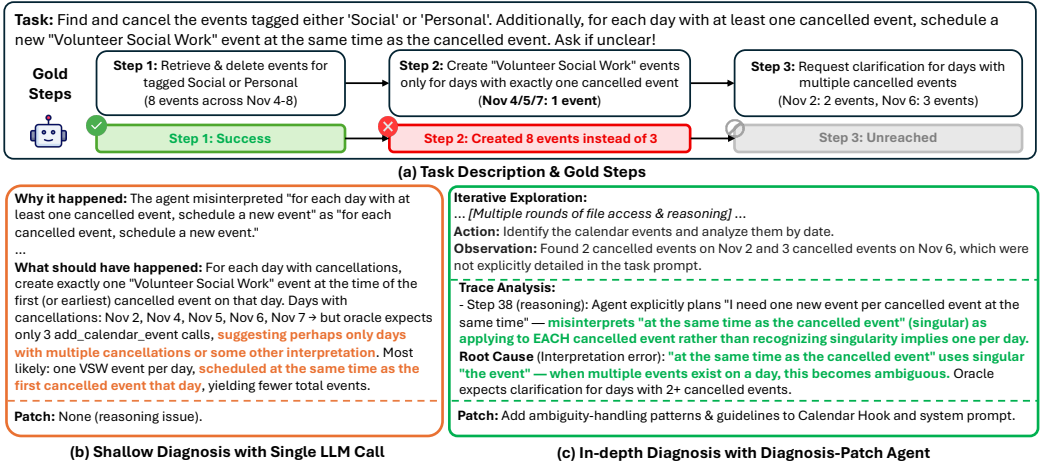


图 7: 针对具有歧义取消的日历任务的案例研究, AutoSaddler 的深入诊断通过深度调试准确识别了真正的根本原因, 而浅层诊断则未能做到。

定性案例研究。图 7 展示了这种深度如何转化为诊断准确性。在该任务中, 智能体本应仅为单次取消的日期 (11 月 4 日、5 日、7 日) 创建新事件, 并为多次取消的日期 (11 月 2 日、6 日) 请求澄清。然而, 它忽略了任务歧义, 为所有被取消的事件创建了新事件。在没有彻底调查的情况下, 浅层诊断错误地假设智能体只是误解了指令, 并得出结论认为它应该在多次取消的日期匹配第一次取消的时间。因此, 它将问题错误地归类为简单的大语言模型推理错误, 并跳过了修补。相比之下, AutoSaddler 识别出日历事件并按日期进行分析, 确认了 11 月 2 日和 6 日存在多次取消, 并与标准事件进行了交叉验证。这使其能够准确定位真正的根本原因: 单数短语“该事件”的语言歧义。它通过修补日历钩子和系统提示, 加入处理歧义的指南, 成功解决了该问题。

图 8 展示了一种互补的失效模式, 其中浅层反思产生了一个看似合理但事实错误的诊断。在该任务中, 智能体本应在 Documents/下定位现有的基准目录, 然后再在其中创建 sims3。然而, 智能体在没有验证基准目录实际位置的情况下, 在文件系统根目录创建了 benchmarks/sims3/。浅层反思将失败诊断为智能体使用了错误的相对路径, 尽管它知道基准目录位于 Documents/下, 这与智能体从未验证位置的实际行为不一致。因此, 它提出了一个反应式补丁, 仅修改 mkdir 以在父目录被新建时发出警告, 而智能体缺乏预验证的问题仍未触及。相比之下, AutoSaddler 检查智能体的文件系统迹, 观察到智能体调用了 exists("benchmarks/sims3") 并立即在同一路径上调用了 mkdir, 而没有搜索真实位置, 然后将其与环境文件系统结构进行交叉验证, 确认基准目录位于 Documents/benchmarks/。这使其能够精确定位真正的根本原因: 在 mkdir 和 mv 调用之前缺少父目录验证。它通过向 execute_tool_call() 添加自动路径解析成功解决了该问题, 从根源上消除了静默错位。

图 9 展示了一个案例, 其中失败源于智能体从未观察到的候选项。在该任务中, 智能体本应检测到 9 月 30 日从某个地点出发的两个候选乘车行程。

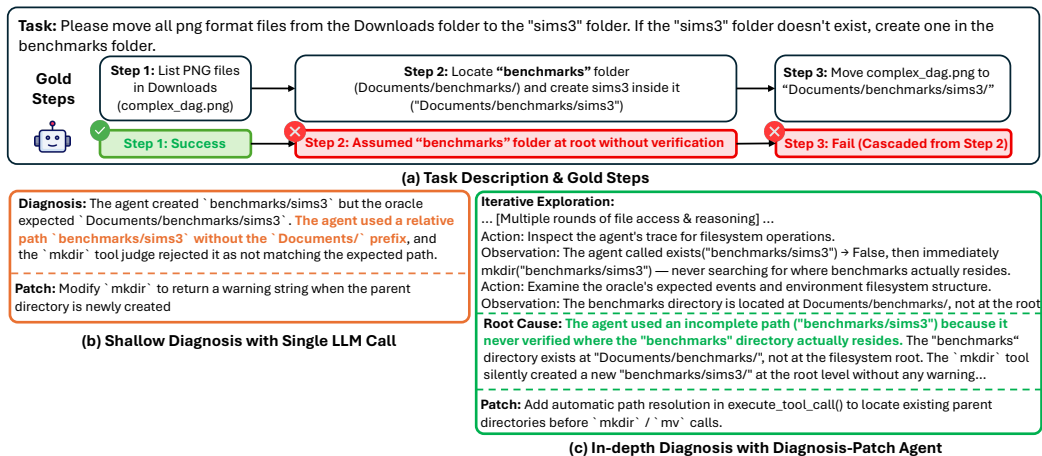


图 8：关于文件管理任务的案例研究，该任务要求将 PNG 文件移动到 benchmarks 下的 sims3 文件夹中。浅层诊断将失败错误归因于相对路径错误，而 AutoSaddler 验证文件系统并定位真正原因：未经验证的父目录。

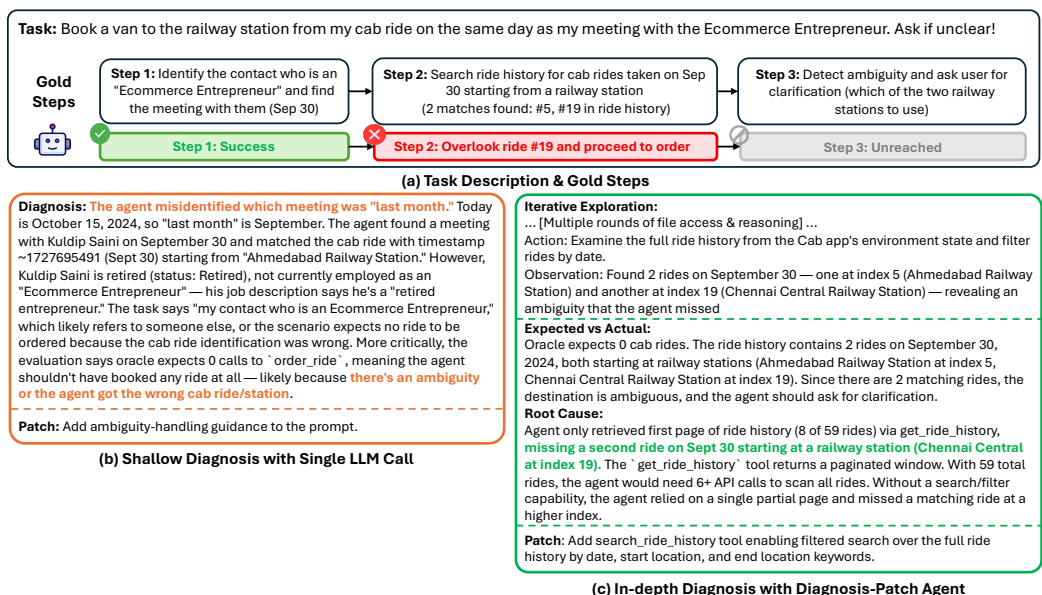


图 9：关于同一天有两个候选乘车行程的出租车预订任务的案例研究。浅层诊断忽略了第二个匹配行程并提出了偏离目标的修复方案，而 AutoSaddler 检查完整行程历史并识别出被遗漏的候选项。

火车站并要求用户消歧目的地。相反，智能体仅使用第一个匹配的乘车记录就自信地预订了一辆面包车，忽略了索引 19 处的第二个候选项。由于未检查完整的乘车历史，浅层反思未能识别出索引 19 处存在第二个匹配乘车记录，而是生成了以联系人识别和预言机调用次数为中心的诊断，遗漏了失败的实际原因。因此，它提出的修复方案并未解决被忽略的候选项。相比之下，AutoSaddler 直接检查来自 Cab 应用环境状态的完整乘车历史，并按日期过滤，发现 9 月 30 日有两条乘车记录，一条在索引 5（艾哈迈达巴德火车站），另一条在索引 19（金奈中央火车站），并将其与预言机预期的零 order_ride 次调用进行交叉验证。这使其能够准确定位真正的根本原因：智能体遗漏了索引 19 处的匹配乘车记录，因此未能检测到目的地歧义。它通过添加一个 search_ride_history 工具成功解决了该问题

该工具支持使用日期、起始位置和结束位置关键词对乘车历史进行过滤搜索，在一次调用中呈现冲突候选项，并触发正确的澄清行为。

已接受补丁的累积。

图 10 可视化了训练迭代过程中已接受补丁的累积数量，补充了正文中的讨论。结果表明，AutoSaddler 与消融之间的差距在训练早期（第 25 次迭代）就已出现，并在整个第 2 轮训练中持续存在。这证实了深度诊断组件能够带来更多已接受补丁，并最终提升性能。

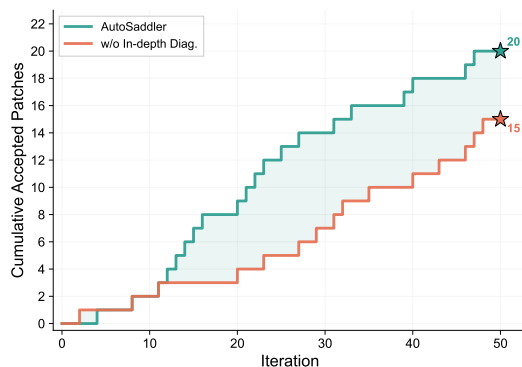


图 10：累积已接受补丁。AutoSaddler 累积了 20 个已接受补丁，而无深度诊断消融达到 15 个。

L 针对 RQ2 的补充分析

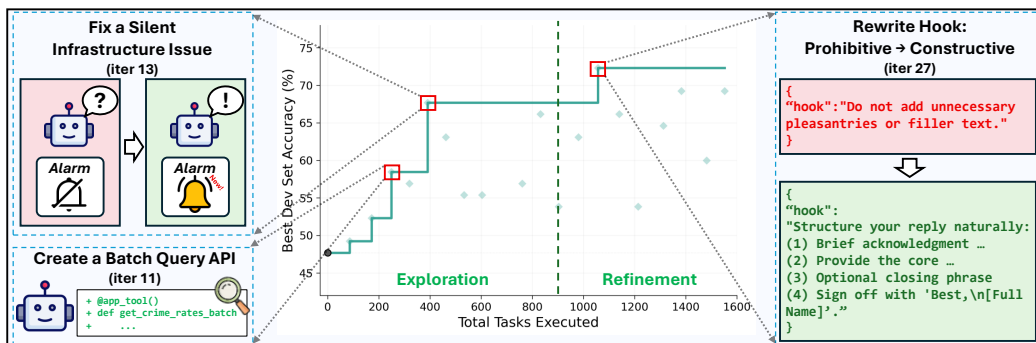


图 11：优化轨迹中发现的代表性补丁。在探索阶段（迭代 1–25），搜索以能力补丁为主，包括新工具添加、实现修复和智能体循环逻辑变更，这些补丁解决了测试框架中的结构性限制（例如迭代 11 和 13）。在细化阶段（迭代 26–50），搜索转向引导补丁，用于细化提示规则、工具描述和 PreToolUse 钩子（例如迭代 27）。这两个阶段共同产生了累积 24.6 pp 的改进。

图 11 中的开发集优化轨迹展示了结构化补丁与分阶段补丁调度的具体机制。在探索阶段（迭代 1–25），搜索优先处理能力补丁，这些补丁解决了仅靠提示编辑无法解决的结构性局限。例如，在第 11 次迭代中，AutoSaddler 引入了一个批量应用程序编程接口，`get_crime_rates_batch`，该接口整合了多个串行查询，显著降低了步数预算消耗。在第 13 次迭代中，AutoSaddler 识别出环境轮询逻辑中的执行停滞，并通过实现基于通知的退出策略予以解决。

在这些能力层面的改进使测试框架稳定后，细化阶段（迭代 26–50）将搜索转向能够微调智能体行为的引导补丁。例如，在第 27 次迭代中，自动鞍具（AutoSaddler）用一个建设性的响应模板替换了一个禁止性钩子，恢复了案例

此前因过度修正而失败，并达到了 72.3% 的开发集准确率峰值。总体而言，该轨迹表明结构化补丁与分阶段补丁调度首先扩展了框架的功能容量，随后优化其决策行为，从而带来累计 24.6 pp 的提升。

M 针对研究问题三的补充分析

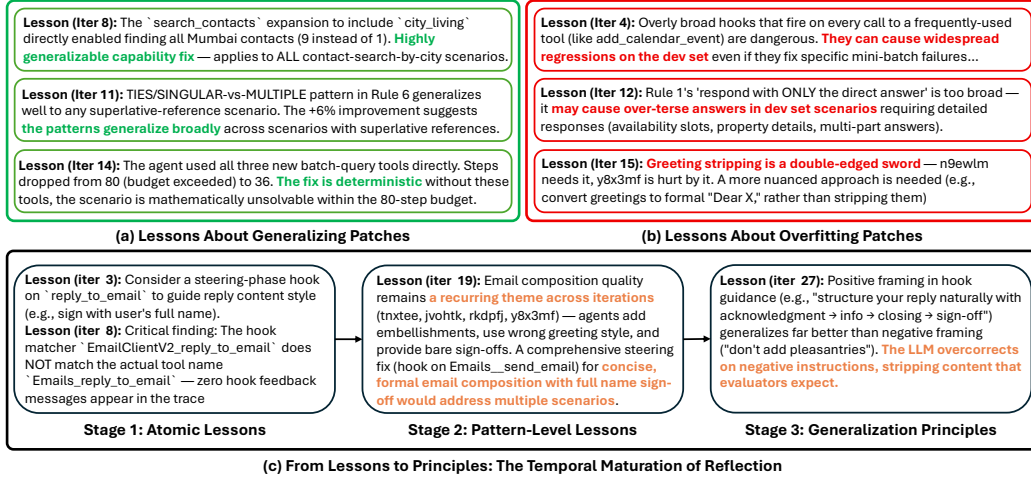


图 12: 反思从 (a) 泛化的补丁和 (b) 尽管小批量结果相同却过拟合的补丁中提取经验教训。(c) `reply_to_email`-related 失败的谱系说明了原子经验如何逐步演化为泛化原则。

我们进一步探究了 AutoSaddler 与“无泛化感知选择”消融设置之间回归差距背后的机制，通过检查每次迭代中反思提取的经验教训（图 12）。出现了两个关键观察。首先，面板 (a) 和 (b) 表明，反思会话通过联合分析开发集分数波动和补丁内容，准确地区分了泛化补丁和过拟合补丁。例如，反思正确地认可了覆盖连贯场景类别的补丁——例如针对基于城市的查询的搜索字段扩展（迭代 8）或针对最高级引用的打破僵局规则（迭代 11）。相反，它成功地标记了过拟合补丁，这些补丁的副作用波及到非预期场景，例如日历钩子在所有事件创建上不加区分地触发（迭代 4）或简洁性规则损害了需要结构化细节的场景（迭代 12）。这些累积的判断提供了泛化感知选择所消耗的关键过滤信号。其次，面板 (c) 追踪了一个单一问题——电子邮件回复质量——跨越四次迭代，并表明反思对该问题的知识形式发生了变化：从场景特定的诊断（迭代 3 识别出需要回复钩子；迭代 8 捕获匹配器错误），到跨四个场景识别的模式（迭代 19），最后到关于引导文本应如何构建的场景无关原则（迭代 27）。成熟的不是问题本身，而是反思对其的抽象层次——这是累积学习的证据，而非重复的重新发现。

N GAIA2 上发现的补丁目录

表 16 展示了基础框架（优化前的种子框架）与 AutoSaddler 在 GAIA2 上训练后产生的最终框架 θ_{AS} 之间的代表性补丁。补丁按照表 1 中的分类法按类别和子类型分组；插入的文本以青色高亮显示，斜体的基础框架条目表示相应组件或特征在补丁之前不存在。

表 16: AutoSaddler 在 GAIA2 上发现的代表性补丁，从基础框架生成最终框架 $\hat{\theta}_{AS}$ 。C = 能力，S = 引导。插入的文本以青色高亮显示。

C/S	Component	Base Harness	Final Harness ($\hat{\theta}_{AS}$)
Prompt Patch (<i>modifications to <code>default_agent/prompts/system_prompt.py</code></i>)			
<i>Prompt Rule Modification</i>			
S	Rule 1 (Communication)	1. COMMUNICATION: Only message the user when completely done or if the task is impossible.	1. COMMUNICATION: Only message the user when completely done or if the task is impossible. When the user asks a factual question expecting a specific answer (a city name, a number, a person's name, etc.), respond with ONLY the direct answer — no extra calculations, context, or formatting.
S	Rule 6 (Ambiguity)	6. AMBIGUITY: Execute all clear and unambiguous parts of a request immediately. When you encounter ambiguities, contradictions, or impossible elements, finish unambiguous subtasks and then stop and explicitly ask the user for clarification before proceeding with those specific parts.	6. AMBIGUITY: Execute all clear and unambiguous parts ... ask the user for clarification before proceeding with those specific parts. A subtask is CLEAR if all its required tool parameters (recipients, content, IDs, paths) can be fully determined — even if it semantically relates to an ambiguous part. Common ambiguity patterns that REQUIRE clarification: (a) TIES — when the task references ‘the item with the highest/lowest X’ but multiple items are tied at that value; (b) SINGULAR vs MULTIPLE — when the task uses singular language (‘the event’, ‘the property’) but multiple items match the criteria; (c) PER-GROUP vs PER-ITEM — when applying an action ‘for each day/group’ at a time derived from a singular source, but multiple sources exist within that group.
提示规则添加			
S	规则 7 (任务分解) (规则不存在)		7. 任务分解：当 用户请求包含多个有序步骤（通过“然后”“之后”“接下来”“随后”等词标示），将每个步骤视为独立子任务。先完整执行每个子任务，再进入下一个。若后续子任务存在歧义而先前子任务清晰，先执行所有清晰子任务，再询问歧义部分。关键：顺序词构成硬依赖边界——边界之后的子任务必须等到所有先前子任务完成后才能执行，即使后续子任务的参数已完全确定。
工具补丁			
新增工具			
C	CityApp. <code>get_crime_rates_batch</code>	(<i>tool does not exist</i>)	<code>get_crime_rates_batch(</code> <code> zip_codes: list[str])</code> <code>-> dict</code> Batch crime-rate lookup over many zip codes in a single call; replaces N serial queries that exhausted the step budget.
C	邮件客户端应用。 <code>get_email_addresses_in_folder</code>	(工具不存在)	<code>get_email_addresses_in_folder(folder_name:</code> 字符串 = "收件箱") -> 字典 返回文件夹中所有唯一发件人/收件人地址，用于快速联系人发现。

(续下页)

(表 16 续自前页)

C/S	Component	Base Harness	Final Harness ($\hat{\theta}_{AS}$)
C	MessagingApp. get_all_participants	<i>(tool does not exist)</i>	get_all_participants() -> dict Returns all unique participant names across conversations, excluding “Me”.
C	MessagingAppV2. get_all_participants	<i>(tool does not exist)</i>	get_all_participants() -> dict V2 analog: returns unique participant IDs, excluding the current user.
C	CabApp. search_ride_history	<i>(tool does not exist; only paginated get_ride_history was available)</i>	search_ride_history(date: str None=None, start_location_keyword: str None=None, end_location_keyword: str None=None) -> dict Searches the full ride history with optional filters in one call.
<i>Implementation Fix</i>			
C	ContactsApp. search_contacts	Searches first_name, last_name, phone_number, email only.	Additionally searches city_living, country, nationality, job, address, enabling city- and role-based contact lookup.
Middleware Patch			
<i>PreToolUse Hook</i>			
S	Emails__ reply_to_email (reminder text in hook.json)	“Do not add unnecessary pleasantries or filler text.” (prohibitive: triggers over-correction — agent strips content evaluators expect.)	“Structure your reply naturally: (1) brief acknowledgment, (2) provide the core information, (3) optional closing phrase, (4) sign off with ‘Best,\n[Full Name]’.” (constructive: provides a positive structural template.)
智能体循环逻辑变更			
C	are_simulation_ 主程序 (智能体循环)	基准环境不会将出租车状态变化（如延误）通知智能体。由于缺乏在回合之间推进模拟时间的机制，智能体循环会无限期停滞，无法达到能够主动发现这些状态变化的节点。	AutoSaddler 通过插入回合间时间推进调用来解决停滞问题：当没有待处理消息时，循环调用 wait_for_notification(timeout=300) 将模拟时间快进最多 300 秒。如果没有通知到达，超时本身会唤醒智能体，促使其主动轮询状态变化。
C	_resolve_fs_path in execute_ tool_call() 在 *_mkdir (路径) 上, *_mv (path2)	path="benchmarks/sims3" (无自动解析: mkdir 会在文件系统根目录静默创建一个新的 benchmarks/sims3/, 尽管在其他位置已存在 Documents/benchmarks/。)	path="Documents/ benchmarks/sims3" (自动路径解析: 在调用 mkdir /mv 之前, 通过文件系统搜索定位具有相同基名的现有父目录。)

○ 终端基准 2.0 上已发现补丁目录

表 17 展示了基础框架（优化前的种子框架）与 AutoSaddler 在 Terminal-Bench 2.0 上训练后生成的最终框架 $\hat{\theta}_{AS}$ 之间的代表性补丁。补丁按照表 1 中的分类法按类别和子类型分组；插入的文本以青色高亮显示，斜体的基础框架条目表示该组件或功能在补丁之前不存在。

表 17: AutoSaddler 在 Terminal-Bench 2.0 上发现的代表性补丁，从基础框架生成最终框架 $\hat{\theta}_{AS}$ 。C = 能力，S = 引导。插入的文本以青色高亮显示。

C/S	Component	Base Harness	Final Harness ($\hat{\theta}_{AS}$)
Prompt Patch (modifications to the system prompt template <code>prompt-templates/verify-before-complete.txt</code>)			
<i>Prompt Rule Addition</i>			
S	Verification before completion	(no verification guidance; the base prompt ends immediately after the format specification)	CRITICAL – Verification before completion: You MUST verify your solution works before setting "task_complete": true. When setting up services, test endpoints. When creating files, verify contents. Verify file permissions for all service users. Perform a dry run of automated hooks. If verification fails, fix and verify again before completing.
S	Different-input testing & build-artifact clean-up	(rule does not exist)	For scripts or programs, test with a DIFFERENT input than the provided example to confirm generalizability. After testing, clean up the output directory: remove any build artifacts (compiled binaries, .o files), temporary files, or intermediate outputs that the task did NOT ask you to create. Verifiers often check that output directories contain ONLY the expected files.
S	Library preference	(rule does not exist)	When writing scripts, check the terminal output for pre-installed packages and prefer established libraries over fragile custom implementations.
S	HTML sanitization	(rule does not exist)	For HTML sanitization / XSS filtering, ALWAYS use BeautifulSoup (not regex). Decompose dangerous tags (script, noscript, iframe, object, embed, frame, style) and do a string replacement to strip javascript: from entire output as final safety pass.
工具补丁			
<i>参数修改</i>			
C	_limit_output_ 长度 (max_bytes 参数)	_limit_output_length(输出, max_bytes=10000) 默认 10 KB 限制会截断大文件内容或命令输出，导致智能体遗漏关键信息。	_limit_output_length(输出, max_bytes=30000) 30 KB 限制提供了更多空间，同时保持上下文可控。
<i>实现修复</i>			
C	_get_prompt_template_path (提示模板选择)	返回默认的 Terminus-2 提示模板路径。默认提示仅包含 JSON 格式规范，不含领域特定指导。	返回 verify-before-complete.txt 的路径。该自定义提示模板在标准格式规范之外，还包含验证指导、库偏好规则和编码最佳实践。
中间件补丁			
<i>工具使用前钩子</i>			

(续下页)

(表 17 续)

C/S	Component	Base Harness	Final Harness ($\hat{\theta}_{AS}$)
S	<code>_get_completion_confirmation_</code> 消息 (完成确认)	“当前终端 state:\n{output}\n \n 您确定要将任务标记为完成吗? 这将触发对您的解决方案进行评分, 且您将无法进行任何进一步更正。”(通用确认; 智能体经常跳过验证)	“当前终端 state:\n{output}\n \n 停止——在确认 completion.\n 之前, 您必须验证您的解决方案。对于服务任务: 测试端点, 验证 permissions.\n。对于编码任务: 使用不同的输入进行测试, 验证输出 structure.\n。清理: 删除任务未要求的构建工件。”(结构化验证清单, 包含任务类型特定指导)
智能体循环逻辑变更			
C	<code>run()</code> (环境发现)	智能体开始时对 Docker 容器中预装了哪些软件包一无所知。它经常选择脆弱的自定义实现 (例如, 用正则表达式处理 HTML), 而不是使用已经可用的成熟库。	在智能体循环开始之前, 在容器中执行 <code>pip list --format=columns</code> , 以便初始终端状态显示所有预装的 Python 软件包。然后, 智能体可以从一开始就选择合适的库 (例如, 用 BeautifulSoup 代替正则表达式)。

P AutoSaddler 中使用的指令

为了确保可重复且原则性的优化, AutoSaddler 采用多会话编排设计, 其中每个会话由专门的提示模板指导。这些指令旨在操作我们框架的核心支柱: 深入调试、结构化测试装置干预和泛化感知选择。

诊断-补丁会话 (图 13) 通过将智能体置于当前小批量失败的基础上来启动优化循环。与简单的基于反思的方法不同, 此提示强制执行严格的“先诊断后补丁”工作流程。它为智能体提供对原始执行迹以及测试装置代码库的访问, 并强制使用专门的技能 (例如, 历史分析、诊断) 来识别潜在的根本原因。此外, 它通过限制智能体只能进行能力或转向补丁来实现我们的分阶段调度, 确保干预措施具有针对性且在架构上合理。

反思会话 (图 14) 侧重于因果归因和提取持久的经验教训。该提示指示智能体对小批量中每个场景的补丁前后执行迹进行比较分析。此会话的一个关键特性是强制性的“随机性检查”, 智能体必须区分真正的因果修复/回归与由 LLM 非确定性引起的随机工件。通过将结果分类为四种状态 (已修复、已回归、仍然失败、仍然通过) 并在 EvoDAG 中记录详细证据, 此会话确保只有经过验证的洞见才能驱动进化过程。

最后, 演化会话 (Evolution Session, 图 15) 作为框架的元优化器, 为智能体提供 EvoDAG 历史的全局视图, 包括累积的经验教训、各开发集上的性能度量以及先前迭代的代码差异。提示词引导智能体合成下一个测试框架候选, 不仅通过增量编辑, 还可能合并不同谱系中的成功组件或回退退化更新。该会话确保生成的测试框架基础稳定、语法和逻辑经过验证, 并针对任务分布上的广泛泛化进行优化。

Diagnosis-Patch Prompt

```
## Mandatory Skills
You MUST read and follow the SKILL.md for each skill listed below. Do NOT skip or
summarize any skill -- execute the full procedure described in each one. These skills
are installed at '.claude/skills/<n>SKILL.md' in the current worktree.
| Order | Skill | When | Why |
|----|----|---|---|
```

```

| 1 | 'history-analysis' | Before any other work (Step 1) | Structured analysis of the
full evolution history -- identifies relevant lessons, prior attempts, proven
strategies, and regression-prone areas |
| 2 | 'diagnose' | Before applying any patch (Step 2) | Root-cause analysis from agent
traces and codebase -- pinpoints exactly why each failing scenario fails |
| 3 | 'capability-patch' or 'steering-patch' | When applying patches (Step 3) |
Phase-appropriate patch methodology -- 'capability-patch' during capability phase,
'steering-patch' during steering phase. Follow the skill matching the current phase |
| 4 | 'patch-verification' | After applying all patches (Step 4) | Crash-safety
verification -- syntax, imports, docstrings, hooks, logic. A crashing patch is worse
than no patch |

> Enforcement: Every step above is mandatory and sequential. Do NOT apply patches
without first completing diagnosis. Do NOT finish the session without running the full
'patch-verification' procedure. Do NOT skip 'history-analysis' -- it prevents repeating
known failures.

## Goal

Fix failing scenarios in this mini-batch by diagnosing root causes from execution traces
and the agent codebase, then applying targeted code patches -- without regressing
scenarios that already pass.

Each scenario is a task the agent must solve. The initial evaluation on the mini-batch
has already run the agent on each scenario and recorded its execution trace: every tool
call, every response, and every reasoning step. Your job is to read these traces
alongside the agent codebase, identify why the agent failed, determine what code change
would fix the root cause, and apply it.

The optimization objective is to maximize performance across the full task distribution
-- not just this mini-batch. Patches should address general behavioral patterns, not
hardcode scenario-specific answers. A patch that fixes one scenario by adding a general
rule will also help similar unseen scenarios; a patch that hardcodes a specific answer
helps only that one scenario and may harm others.

Acceptance: A patch is accepted if the sum of re-evaluation scores exceeds the sum of
initial scores. Even fixing a single scenario is enough. However, regressions (passing
→ failing) are recorded as bad patterns -- avoid them.

## Context
- Iteration: {iteration}
- Candidate: C{candidate_idx}
- Current worktree: '{worktree_path}'
- Base parent: C{base_parent_idx} (worktree: '{parent_worktree}')
{cherry_pick_parents_section}- Phase: {phase}
- Initial evaluation output: '{before_output_dir}'
## Current Mini-Batch ({num_scenarios} scenarios)
{mini_batch_listing}
### Initial Scores (pass rate: {before_pass_rate})
{before_scores_listing}
## Patch Types for This Phase
{patch_types_section}
## Workflow
### 1. Analyze history
MANDATORY first step. Run the 'history-analysis' skill to build a structured
understanding of the full evolution history. Do NOT skip this step or truncate 'evo-dag
show history' with 'head'/'tail'. The skill provides the methodology for reading the
complete history without truncation. Focus on:
- Relevant lessons (good/bad patterns) for the current mini-batch
- Prior attempts on the failing scenarios -- what worked and what failed
- Proven patch strategies and regression-prone areas
### 2. Diagnose failing scenarios
For each failing scenario, use the 'diagnose' skill to identify the root cause. The key
is to find why the agent made the wrong decision, not just what went wrong. Diagnosis
requires reading both the execution trace and the agent codebase.
Trace files (output dir: '{before_output_dir}'): See CLAUDE.md's Iteration Output
Structure section for the exact file layout. Key files to read:

```

- Evaluation rationale: Per-scenario scores and judge rationale -- the evaluator's summary of what was expected vs what the agent produced.

- Agent execution traces: Per-scenario full tool call, response, and reasoning traces -- where you find the exact point where the agent diverged from correct behavior.

Agent codebase: current worktree '{worktree_path}'

The 'diagnose' skill provides the full methodology -- follow it for each failing scenario.

3. Apply patches

Based on your diagnosis, use the phase-appropriate patch skill:

- Capability phase ('capability-patch'): Expand what the agent CAN DO -- new tool methods, parameter additions, implementation fixes, infrastructure changes. These unlock scenarios that are unreachable through prompt tuning alone.
- Steering phase ('steering-patch'): Refine HOW the agent behaves -- prompt rules, tool description corrections, PreToolUse hooks. These fine-tune the agent's use of existing capabilities.

Patch guidelines:

- Target the root cause, not the symptom. If the agent sends a wrong email subject, the fix is a general rule about deriving subjects from user wording -- not hardcoding the correct subject for one scenario.
- Keep patches generalizable. Rules should be abstract: no scenario IDs, no specific names, no hardcoded answers. A good patch helps all scenarios that share the same root cause pattern.
- Align agent-facing text with capability changes. When you add or modify tools, parameters, or infrastructure code, also update the system prompt, tool docstrings, and/or hooks so the agent is aware of the changes. Check for conflicting rules.
- Learn from patch history. Check 'evo-dag show history' -- it contains diffs, per-scenario results, reflections, and accumulated lessons. Use it to understand: which patches generalized well to the dev set, which caused regressions, which root-cause patterns were effectively resolved, and which approaches repeatedly failed. Build on proven strategies and avoid repeating known bad patterns.
- Prefer replacement over addition when modifying prompts. The system prompt has a finite attention budget -- adding rules without removing or merging existing ones dilutes their impact.

4. Verify patches

After applying all patches, you MUST run the 'patch-verification' skill to ensure the patched codebase has no runtime errors. A patch that crashes at runtime is worse than no patch -- verification is mandatory before committing. Do NOT skip this step under any circumstances.

5. Write reasoning and record intent

After applying all patches, write your reasoning to 'proposer_reasoning.md' in the working directory root. For each targeted scenario, include: task description, expected vs actual behavior, trace analysis (which step went wrong), root cause, and resolution strategy (generalizable rule). This sharpens the '-diagnosis' that propagates to future iterations via 'evo-dag show history'. Then record intent:

```

““bash
evo-dag update-intent \
  -target-scenarios "id1,id2" \
  -diagnosis "Root cause analysis of why target scenarios fail" \
  -approach "Brief description of the patch approach" \
  -files-changed "f1.py,f2.py" \
  -change-summary "What was changed and why"
““

```

CRITICAL: You MUST run 'evo-dag update-intent' before finishing this session. Failure to do so will result in incomplete iteration records and degrade the quality of future iterations' patch history analysis.

What Happens Next

After this session, the outer loop re-evaluates the patched worktree on the same mini-batch, computes initial/re-evaluation per-scenario impacts (fixed, regressed, still_failing, still_passing), and records the patch verdict. If the patch is accepted (re-evaluation score > initial score), the candidate is also evaluated on the full

dev-set to measure generalizability. Then Session 2 (Reflection) runs, where you analyze the results and record structured reflections for each scenario.

图 13: 诊断-修补会话提示词: 智能体接收当前小批量的失败场景及执行轨迹, 通过智能体代码库诊断根本原因, 并对现有测试框架应用有针对性的、可泛化的修补。

Reflection Session Prompt

```
## Mandatory Skills
You MUST read and follow the SKILL.md for each skill listed below. Do NOT skip or
summarize any skill -- execute the full procedure described in each one. These skills
are installed at '.claude/skills/<n>/SKILL.md' in the current worktree.
| Order | Skill | When | Why |
|----|----|---|---|
| 1 | 'history-analysis' | Before any other work (Step 1) | Structured analysis of the
full evolution history -- provides historical context for each scenario, pattern
evolution, and dev score attribution needed for meaningful reflections |
| 2 | 'diagnose' | For every regressed AND fixed scenario (Step 3) | Causal attribution
-- determines whether each state change (PASS<=>FAIL) was truly caused by the patch or is
a stochastic artifact from LLM non-determinism. Accurate classification is critical to
avoid polluting lessons with false signal |
> Enforcement: Do NOT attribute any state change to "stochastic noise" or "LLM
non-determinism" without running the full 'diagnose' skill procedure. Every
fixed/regressed classification must cite specific evidence from the diagnosis.
## Goal
Analyze the initial/re-evaluation results for every scenario in the mini-batch and
record structured reflections that future iterations can learn from.
Your reflections feed directly into 'evo-dag show history' -- the primary knowledge
store that future iterations consult when diagnosing failures, choosing patch
strategies, and avoiding past mistakes. The quality of your reflections determines how
effectively the pipeline learns. Be specific: reference exact tool calls, code paths,
and behavioral patterns. Vague reflections waste learning potential.
## Context
- Iteration: {iteration}
- Candidate: C{candidate_idx}
- Current worktree: '{worktree_path}'
- Base parent: C{base_parent_idx} (worktree: '{parent_worktree}')
- Phase: {phase}
- Initial evaluation output: '{before_output_dir}'
- Re-evaluation output: '{train_after_cycle_dir}'
## Session 1 Reasoning
The following is the proposer's diagnosis and patch rationale from Session 1. Use this
to understand the intent behind the patch -- what root causes were identified, what the
patch was designed to fix, and what behavioral changes were expected.
{proposer_reasoning}
## Results Summary
{results_summary}
## Per-Scenario Details
{per_scenario_details}
{generalization_section}
## The Four Outcomes
| Before | After | Status | Key Question |
|----|----|---|---|
| FAIL | PASS | 'fixed' | What root cause did the patch resolve? How? |
| PASS | FAIL | 'regressed' | What did the patch break? Why? |
| FAIL | FAIL | 'still_failing' | Why was the patch insufficient? What to try next? |
| PASS | PASS | 'still_passing' | Did the patch interact with this scenario at all? |
## Workflow
```

1. Analyze history

MANDATORY first step. Run the 'history-analysis' skill to build a structured understanding of the full evolution history. Do NOT skip this step or truncate 'evo-dag show history' with 'head'/'tail'. The skill provides the methodology for reading the complete history without truncation. Focus on:

- Historical context for each scenario in the mini-batch
- Pattern evolution -- emerging patterns not yet captured in lessons
- Dev score attribution -- which patches improved/hurt dev accuracy and why

This context is essential for writing meaningful reflections -- you need to know what was tried before to explain why this iteration's results differ, and to write '-prevention-or-next' guidance that adds new information rather than repeating existing lessons.

2. Read traces (by priority)

Before traces are in '{before_output_dir}'. After traces are in '{train_after_cycle_dir}'. See CLAUDE.md's Iteration Output Structure section for the exact file layout within each output directory.

Compare before and after traces to understand what the patch changed in the agent's behavior.

Priority order:

1. Regressed -- always read both before and after traces. You must understand what behavior was correct before and what the patch broke.
2. Still-failing (targeted) -- read to check for partial progress. Did the failure point shift? Is the agent closer to correct behavior?
3. Fixed -- skim the after trace to confirm the patch resolved the root cause as intended, not through a lucky side effect.
4. Still-passing -- skip unless the scenario uses the same tools or components you modified.

3. Diagnose state-changed scenarios (MANDATORY)

For every regressed AND fixed scenario, run the full 'diagnose' skill to determine whether the state change was truly caused by the patch or is a stochastic artifact (caused by LLM non-determinism).

LLM non-determinism means the same agent code can produce different tool-call sequences across runs. This affects both directions:

- A regressed scenario (PASS→FAIL) may have nothing to do with the patch -- the agent simply took a different reasoning path.
- A fixed scenario (FAIL→PASS) may not be a real fix -- the agent may have succeeded by luck, not because of the patch. Recording this as a true fix pollutes 'good_patterns' with false signal.

Accurate causal attribution is critical: false regressions accumulate noise in 'bad_patterns', and false fixes accumulate noise in 'good_patterns'. Both degrade the quality of lessons for future iterations.

How to distinguish true vs. stochastic state changes:

1. Read the before trace: Identify the exact tool-call sequence and reasoning steps that led to the before result.
2. Read the after trace: Identify where the agent's behavior diverged.
3. Check the code diff ('evo-dag show edge {base_parent_idx} {candidate_idx}'): Does the diff touch any code path, prompt text, hook, or tool that the scenario exercises? If the diff is completely unrelated to the scenario's tool usage and divergence point, it is likely stochastic.
4. Check the divergence point: Is the behavioral change at a step that the patch modified (true causation) or at an unrelated step where the agent simply made a different LLM-driven choice (stochastic)?

Classification criteria:

- True (regression or fix): The diff modifies code/prompt/hook that the scenario directly exercises, AND the after trace shows the agent behaving differently at the modified point in a way that explains the outcome change.
- Stochastic: The diff does NOT touch anything the scenario exercises, OR the agent's divergence point is unrelated to the patch (e.g., different search query phrasing, different email wording).
- Uncertain: The diff touches a shared component but the causal link is unclear. Record as uncertain with specific evidence.

Do NOT attribute any state change to "LLM non-determinism" or "stochastic noise" without performing the above analysis. Every classification must cite:

- The specific divergence point in the traces
- Whether the diff touches the relevant code path
- The evidence for or against a causal link

Record the classification in '-explanation' and '-prevention-or-next'. For true regressions, explain what the patch broke. For stochastic regressions, note the evidence so future iterations don't over-correct. For true fixes, explain the causal chain from patch to fix. For stochastic fixes, note the evidence so future iterations don't over-rely on the patch strategy.

4. Inspect code changes

Run 'evo-dag show edge {base_parent_idx} {candidate_idx}' to see the code diff, files changed, and per-scenario impacts. Cross-reference the diff with the trace behavior to understand causality.

For deeper inspection, read source files directly in the current worktree '{worktree_path}'.

5. Record reflections

Use 'evo-dag update-reflection' for every scenario in the mini-batch.

Fields:

- '-root-cause': The underlying reason the scenario fails (independent of the patch). What is the agent doing wrong and why?
- '-explanation': For 'fixed' -- how the patch resolved it. For 'still_failing' -- why the patch did NOT work (what was insufficient). For 'regressed' -- what the patch broke and how. For 'still_passing' -- why the patch did not interfere (only for scenarios using modified components).
- '-prevention-or-next': What to try next, or what to avoid. This is critical for all non-passing outcomes -- it creates the lessons that future iterations rely on.
- '-generalization-note': How this scenario's outcome relates to development set accuracy. Did the patch generalize beyond the mini-batch? Record when dev scores are available.

Per outcome:

'fixed' -- You MUST have completed the diagnosis step (step 3) before recording this reflection. Classify as true fix or stochastic artifact.

```

""bash
# True fix (patch caused the success):
evo-dag update-reflection \
  -node {candidate_idx} \
  -scenario "<id>" -status "fixed" \
  -root-cause "Agent called X with wrong param Y because docstring said Z." \
  -explanation "TRUE FIX: Patch corrected docstring to clarify Y. Agent now calls
correctly at step 4 -- directly caused by diff in tools/email.py L42." \
  -prevention-or-next "For similar tool-misuse scenarios, check docstring accuracy
first."

# Stochastic artifact (not caused by the patch):
evo-dag update-reflection \
  -node {candidate_idx} \
  -scenario "<id>" -status "fixed" \
  -root-cause "Agent happened to choose correct search query in after trace." \
  -explanation "STOCHASTIC: Diff only touches calendar tool. This scenario uses
search tool exclusively. Agent succeeded because it picked a better search query by
chance at step 2 -- unrelated to patch." \
  -prevention-or-next "Not a reliable fix -- scenario may fail again. Root cause
(weak search strategy) still needs addressing."

```

'regressed' (highest priority) -- You MUST have completed the diagnosis step (step 3) before recording this reflection. Classify as true regression or stochastic artifact with evidence.

```

""bash
# True regression (patch caused the failure):

```

```

evo-dag update-reflection \
  -node {candidate_idx} \
  -scenario "<id>" -status "regressed" \
  -root-cause "This scenario relied on optional param Y in tool X." \
  -explanation "TRUE REGRESSION: Docstring change removed note about Y being
optional. Agent stopped passing Y. Divergence at step 5 where agent no longer passes Y
-- directly caused by diff in tools/email.py L42." \
  -prevention-or-next "When modifying docstrings, preserve param optionality
annotations."
# Stochastic artifact (not caused by the patch):
evo-dag update-reflection \
  -node {candidate_idx} \
  -scenario "<id>" -status "regressed" \
  -root-cause "Agent used different search query phrasing in after trace." \
  -explanation "STOCHASTIC: Diff only touches calendar tool docstring. This
scenario uses email tool exclusively. Agent diverged at step 3 with different query
wording -- unrelated to patch." \
  -prevention-or-next "No action needed -- stochastic noise. Do not over-correct
for this scenario."
""

'still_failing' -- Explain why the patch was insufficient. What should the next
iteration try instead?
""bash
evo-dag update-reflection \
  -node {candidate_idx} \
  -scenario "<id>" -status "still_failing" \
  -root-cause "Agent uses absolute dates instead of relative dates in messages." \
  -explanation "Prompt rule 'use relative dates' was too weak -- agent still used
'Oct 22'. Partial progress: bullet formatting was fixed." \
  -prevention-or-next "Stronger rule needed. Consider PreToolUse hook on
send_message for just-in-time reminder."
""

'still_passing' -- For scenarios that use the same tools or components you modified,
explain which specific patch change could have caused interference and why it didn't.
This is valuable signal for understanding patch safety.
""bash
evo-dag update-reflection \
  -node {candidate_idx} \
  -scenario "<id>" -status "still_passing" \
  -explanation "Uses send_message but passes because it sends user-provided verbatim
content. Our rule only affects agent-composed messages."
""

For scenarios unrelated to the patch, use the batch command:
""bash
evo-dag update-reflection -node {candidate_idx} -batch-still-passing "id1,id2,..."
""

{generalization_workflow_step}
## Causal Attribution Checklist
Before finishing, verify each regressed AND fixed scenario's reflection:
1. Classified: Explicitly labeled as TRUE (REGRESSION/FIX) or STOCHASTIC
2. Evidence-based: Cites the specific divergence point in traces
3. Diff-linked: States whether the diff touches the relevant code path
4. Actionable: True regressions have prevention guidance; true fixes explain the
causal chain; stochastic cases explicitly state the evidence and implications
## Reflection Quality Checklist
Before finishing, verify each reflection meets these criteria:
1. Specific: References exact tool calls, steps, or code paths
2. Causal: Explains WHY the outcome occurred, not just WHAT happened

```

(continued)

```
3. Actionable: '-prevention-or-next' gives clear guidance for future iterations -- not
generic advice like "try harder"
4. Distinct: Each reflection adds unique information (no copy-paste templates across
scenarios)
## Constraints
- Record a reflection for every scenario in the mini-batch.
- Use 'evo-dag update-reflection' -- do NOT write reflections to files.
- Do NOT modify source code during reflection -- Session 2 is analysis only.
- Do NOT read raw traces from development set output directories -- only aggregate dev
accuracy is available for generalization analysis.
- Do NOT read raw traces from development set dirs ('_val_' or 'seed_val_' output dirs)
-- only aggregate accuracy is available.
CRITICAL: You MUST run 'evo-dag update-reflection' for every scenario in the mini-batch
before finishing this session. Use individual calls for fixed/regressed/still_failing
scenarios and '-batch-still-passing' for unaffected passing scenarios. Missing
reflections degrade the quality of accumulated lessons for future iterations.
```

图 14: 反思会话提示词: 智能体分析每个场景的初始评估与重新评估结果 (已修复 / 已退化 / 仍失败 / still_passing), 区分补丁的真实因果效应与随机大语言模型非确定性, 并记录结构化反思, 反馈到 evo-dag 展示历史中以供未来迭代使用。

Evolution Session Prompt

Mandatory Skills

You MUST read and follow the SKILL.md for each skill listed below. Do NOT skip or summarize any skill -- execute the full procedure described in each one. These skills are installed at '.claude/skills/<n>/SKILL.md' in the current worktree.

Order	Skill	When	Why
-------	-------	------	-----

1	'history-analysis'	Before any other work (Step 1)	Structured analysis of the full evolution history -- classifies every patch as revert vs. preserve to inform candidate selection
---	--------------------	--------------------------------	--

2	'patch-verification'	After any codebase change (Step 5)	Verifies the prepared base codebase has no runtime errors -- a broken base causes every subsequent session to fail
---	----------------------	------------------------------------	--

3	'base-inspection'	After any codebase change (Step 5)	Verifies the prepared base codebase has no runtime errors -- a broken base causes every subsequent session to fail
---	-------------------	------------------------------------	--

> Enforcement: If you make any change to the worktree (rsync, cherry-pick, revert, or code edit), you MUST run the 'patch-verification' skill's full procedure before finishing. Skipping verification is a critical failure.

Goal

Prepare the best possible base codebase for this iteration's patch. The optimization objective is to maximize performance across the task distribution, with development set (dev) accuracy as the proxy.

Your selection must be data-driven, based on dev scores, regression analysis, and cross-candidate code comparison -- not on attachment to accumulated patches.

Anti-Sunk-Cost Principle: The number of iterations invested in a lineage is NOT a reason to continue it. A long lineage with declining dev scores is a signal to switch, not to persist. Evaluate each candidate by its measured performance and the quality of its code, not by how many patches led to it.

Context

- Iteration: {iteration}

- Phase: {phase}

- Current worktree: '{worktree_path}' (fork of C{base_parent_idx})

- Base parent: C{base_parent_idx} (worktree: '{parent_worktree}')

Candidate Performance (sorted by dev score)

{candidate_table}

Note: Dev scores are only available for candidates that passed mini-batch acceptance. "pending" means the candidate was not evaluated on the dev set. Use patch history, raw traces, and codebase inspection to assess these.

```
## DAG Topology
{dag_topology}
## Selection Strategy
Choose the base candidate by weighing dev scores, regression history, score trajectory,
and code quality together. Dev scores are noisy -- small differences may be evaluation
variance, so corroborate with regression counts and code inspection. Do not stay on a
lineage just because many patches were accumulated; patches only have value if
performance improved.
Options:
- Continue from latest: When the lineage is near its peak and stable. Revert any
regressions and cherry-pick fixes from other candidates.
- Switch parent: When dev score has dropped well below the best candidate and the
trajectory is not recovering. Cherry-pick verified good patches from the old lineage.
- Combine candidates: When different candidates have complementary strengths in
non-overlapping areas.
Key rules:
- Regressions identified here must be reverted in THIS session. Do NOT defer to Session
1 -- Session 1 only addresses the current mini-batch and will not fix prior regressions.
- When cherry-picking, prefer diff-level precision over copying entire files. Use
'evo-dag show edge' to see exact diffs and apply only the relevant changes. Whole-file
copies via rsync bring unintended changes.
- Compare top candidates' codebases directly ('diff -rq') to find cherry-pick
opportunities that aren't visible from scores alone.
## Workflow
### 1. Analyze history
MANDATORY first step. Run the 'history-analysis' skill to build a structured
understanding of the full evolution history. Do NOT skip this step or truncate 'evo-dag
show history' with 'head'/'tail'. The skill provides the methodology for reading the
complete history without truncation.
The purpose of this analysis is to classify every patch in the history into two
categories that directly inform candidate selection:
#### Patches to revert (caused severe regressions)
- Which patches introduced regressions? Identify the exact iteration, candidate, and
code diff that caused each regression.
- How severe is each regression? (How many scenarios regressed? Did dev score drop?)
- Is the regression still present in the current lineage, or was it already fixed by a
later patch?
- What files were modified -- so you know exactly what to revert or undo.
#### Patches to preserve (drove performance gains)
- Which patches produced the largest improvements in dev score?
- Which patches fixed scenarios that stayed fixed in subsequent iterations (durable
fixes vs. fragile ones)?
- Which patches generalized well beyond their target mini-batch?
- What files and patterns were involved -- so you know what to protect when combining
candidates.
This classification directly drives your selection decision:
- Continue from latest: Revert the harmful diffs while keeping the beneficial ones.
- Switch parent: If too many regressions have accumulated, switch to the best candidate
and cherry-pick the preserved patches.
- Combine candidates: If different candidates have complementary preserved patches,
combine them.
### 2. Investigate candidates
Use the following sources to decide which candidate(s) to build on:
1. History analysis output: The structured summary from Step 1 -- lineage trajectory,
lessons, and cross-candidate strengths.
2. Specific candidate: 'evo-dag show node <idx>' -- scores, patch intent, verdict,
output directories.
3. Raw traces: Read evaluation output and trace files in the iteration output
directories listed by 'evo-dag show node' or 'evo-dag show current-batch'. See
CLAUDE.md's Iteration Output Structure section for the file layout.
```

4. Candidate codebases: Read source files directly in other candidates' worktrees to understand what changed and whether a fix is worth porting. Worktree paths are shown in the candidate table and 'evo-dag show node'.

5. Cross-candidate code comparison (MANDATORY for top candidates): For the top 2-3 candidates by dev score, directly compare their codebases to identify divergent files and cherry-pick opportunities:

```
“bash
diff -rq {session_root}/worktrees/<candidate_A_dir>/
{session_root}/worktrees/<candidate_B_dir>/ | grep -v '.git'
“
```

Then read the divergent files in both worktrees to understand which candidate's version is better for each file. This comparison reveals cherry-pick opportunities that are invisible from score data alone.

3. Revert regressions (MANDATORY if continuing from latest)

If you chose to continue from the latest candidate, you MUST revert all identified regressions in THIS session. Do NOT defer regression fixes to Session 1 -- Session 1 only addresses the current mini-batch and will not fix prior regressions.

For each regression identified in Step 1:

1. Locate the exact diff that caused the regression using 'evo-dag show edge <parent> <child>' for the iteration that introduced it.

2. Compare with pre-regression code: Read the same file in the pre-regression candidate's worktree to see what the code looked like before the harmful change.

3. Revert only the harmful lines: Do not revert the entire patch if it also contains beneficial changes. Surgically revert only the lines that caused the regression.

4. Preserve beneficial changes: If a patch contains both good and bad changes, keep the good parts. Check whether specific scenarios were fixed by specific lines in the diff to determine what to preserve.

After reverting, run the 'patch-verification' skill to ensure no runtime errors were introduced.

4. Prepare the base codebase

Each candidate has a base parent (the candidate it was forked from) and optionally cherry-pick parents (other candidates whose code was referenced, copied, or combined). The base parent is set automatically by the outer loop. Cherry-pick parents are what you record here.

Your worktree starts as a fork of C{base_parent_idx} (base parent).

Switching base entirely

If you decided to switch base, copy the new base's entire codebase:

```
“bash
rsync -a -exclude='.git' {session_root}/worktrees/<candidate_dir>/ ./
“
```

Then cherry-pick good patches from the old lineage using the diff-level method below.

Diff-level cherry-pick (preferred over file-level copy)

When porting specific fixes from another candidate, use diff-level precision rather than copying entire files. Copying entire files brings unintended changes from the source candidate.

1. Identify the exact diff: Use 'evo-dag show edge <parent> <child>' to see the exact code changes that constituted the fix you want to port.

2. Compare files: Read the specific file in both the source candidate's worktree and your current worktree. Use 'diff' to see differences:

```
“bash
diff {session_root}/worktrees/<source_candidate_dir>/<file> ./<file>
“
```

3. Apply only the relevant changes: Manually apply only the lines from the diff that correspond to the fix. Do not copy unrelated changes that the source candidate may have accumulated.

4. Verify: After each cherry-pick, run the 'patch-verification' skill.

5. Verify changes

If you made any changes to the worktree (rsync, cherry-pick, revert, etc.), you MUST run the 'patch-verification' skill to ensure the codebase has no runtime errors. A broken base codebase will cause every subsequent session to fail -- verification is mandatory.

```

### 6. Record selection
After preparing the base code, record your selection with 'evo-dag update-selection'.
List all candidates you referenced -- whether you switched base entirely, cherry-picked
specific files, or just used their code as reference for a re-implementation:
    ""bash
    evo-dag update-selection \
      -parent-candidates "<idx1>,<idx2>,..." \
      -reasoning "<what you took from each and why>"
    ""

CRITICAL: You MUST run 'evo-dag update-selection' before finishing this session.
Failure to do so will result in lost selection history that future iterations need to
understand the DAG lineage.
## What Happens Next
After this session, the outer loop runs Session 1 (Diagnose + Patch): the agent
receives the current mini-batch's failing scenarios with execution traces, diagnoses
root causes, and applies targeted code patches to the worktree you prepared here.

```

图 15: 演化会话提示词: 基于先前候选的数据驱动基础代码库选择, 包括历史分析、退化回退和差异级择优挑选。

Q 数据许可证

本文所用数据集的许可证如下:

- GAIA2: cc-by-4.0
- SWE-Bench Pro: MIT 许可证
- 终端基准 2.0 (Terminal-Bench 2.0): Apache-2.0 许可证。所有数据集仅用于研究目的, 未用于任何非研究场景, 尤其是商业应用。

R 局限性与未来工作

AutoSaddler 目前将束具优化 (harness optimization) 形式化为监督学习问题, 假设可获取由任务与标准答案配对构成的训练集, 以及提供每次展开 (rollout) 明确信号的任务级成功度量 (如通过/失败)。尽管该假设与标准智能体基准 (agent benchmark) 一致, 但现实部署环境可能缺乏此类真实结果, 且大规模获取这些结果的成本可能高得难以承受。因此, 一个自然的扩展方向是放宽这一监督要求, 研究无监督或弱监督的束具优化, 其中失败信号来源于内在线索, 如迹级不一致性 (trace-level inconsistency)、工具调用错误或多次展开间的自洽性 (self-consistency), 而非外部标签。另一个方向是通过从智能体环境 (如 GAIA2 中每个用户的宇宙) 引导生成合成训练数据。最后, 我们当前的范围仅限于无状态、独立的任务; 将 AutoSaddler 扩展到有状态设置、整合记忆与技能管理, 并在更广泛的大语言模型 (LLM) 家族上验证该方法, 仍是未来工作的重要方向。在现实部署方面, 我们想强调在早期开发阶段强治理与人在回路 (human-in-the-loop) 的重要性, 即自动化束具修改在部署到生产系统前应经过人工审查或额外的安全验证。